MAJOR PROJECT

ON

"TEMPORAL ATTENTION MULTI-SENSOR VISION NETWORK FOR BICYCLE ACCIDENT FAULT DETECTION"

Submitted

In the partial fulfilment of the requirements for

The award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE & ENGINEERING

By

K.SARIKA	**[22U51A0561]**
K.AJAY	**[22U51A0574]**
K.LAHARI	**[22U51A0565]**
E.KANYAKUMARI	**[22U51A0540]**

Under the Esteemed Guidance of

Dr. Gnaneswara Rao Nitta

Professor and Principal

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

DRK COLLEGE OF ENGINEERING AND TECHNOLOGY

MAJOR PROJECT PHASE-II

ON

"TEMPORAL ATTENTION MULTI-SENSOR VISION NETWORK FOR BICYCLE ACCIDENT FAULT DETECTION"

Submitted

In the partial fulfilment of the requirements for

The award of the degree of

BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE & ENGINEERING
By

K.SARIKA	**[22U51A0561]**
K.AJAY	**[22U51A0574]**
K.LAHARI	**[22U51A0565]**
E.KANYAKUMARI	**[22U51A0540]**

Under the Esteemed Guidance of
Dr. Gnaneswara Rao Nitta
Professor and Principal

2025-2026

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

DRK COLLEGE OF ENGINEERING AND TECHNOLOGY

Affiliated to JNTU HYDERABAD Bowrampet,

HYDERABAD-500043

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
DRK COLLEGE OF ENGINEERING AND TECHNOLOGY
Affiliated to JNTUH HYDERABAD,
Bowrampet, HYDERABAD-500043.

CERTIFICATE

This is to certify that the Project Report entitled "**TEMPORAL ATTENTION MULTI-SENSOR VISION NETWORK FOR BICYCLE ACCIDENT FAULT DETECTION**" that is being submitted by **K.Sarika (22U51A0561), K.Ajay (22U51A0574), K.Lahari (22U51A0565), E.Kanyakumari (22U51A0540)**, in partial fulfillment for the award of B.Tech degree in Computer Science and Engineering to the **DRK COLLEGE OF ENGINEERING AND TECHNOLOGY Affiliated to JNTU HYDERABAD**, is a record of bonafied work carried out by them under the supervision of faculty member of CSE Department.

Guide External Examiner HOD_CSE

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
DRK COLLEGE OF ENGINEERING AND TECHNOLOGY Affiliated to
JNTUH HYDERABAD,
Bowrampet, HYDERABAD-500043.

DECLARATION

We hereby declare that the MAJOR PROJECT entitled "**TEMPORAL ATTENTION MULTI SENSOR VISION NETWORK FOR BICYCLE ACCIDENT FAULT DETECTION**" submitted for the DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING. This dissertation is our original work and the PROJECT has not formed the basis for the award of any degree, associate-ship, and fellowship, or any other similar titles, and no part of it has been published or sent for publication at the time of submission.

By:

K.SARIKA	**[22U51A0561]**
K.AJAY	**[22U51A0574]**
K.LAHARI	**[22U51A0565]**
E.KANYAKUMARI	**[22U51A0540]**

Date: 20-04-2026

ACKNOWLEDGEMENT

The completion of this Major Project has been a valuable opportunity for personal growth and academic development. We consider ourselves fortunate and honored to have had the guidance and support of many esteemed individuals throughout this endeavour.

We wish to extend my profound gratitude and sincere appreciation to our guide and esteemed principal**, Dr. Gnaneswara Rao Nitta** sir, for his unwavering support and insightful guidance throughout the project. His expertise and encouragement were instrumental in the successful completion of this work.

We would like to express our heartfelt thanks to **Dr. Madhuri**, Head of the Department of Computer Science and Engineering, for her assistance and for creating an environment conducive to effective work.

We are grateful to the management members, Sri **M. Chandra Sekhara Rao (Director**), **Sri D. Sriram (Treasurer)**, and Honourable Chairman, Sri. **D. B. Chandra Sekhar Rao**, for their unwavering support and resources provided.

We extend our wholehearted gratitude to all the faculty members of the Department of Computer Science and Engineering, whose guidance and instruction have been crucial throughout our academic journey.

With sincere Regards,

K.SARIKA [22U51A0561]
K.AJAY [22U51A0574]
K.LAHARI [22U51A0565]
E.KANYAKUMARI [22U51A0540]

List of Contents

1 Introduction 2
2 LITERATURE SURVEY AND BASE PAPER 5
2.1 Studies on Accident Detection and Transportation Safety Analytics 6
2.2 Handling Class Imbalance in Safety-Critical Classification 6
2.3 Feature Engineering for Categorical and Mixed-Type Data 7
2.4 Attention-Based and Hybrid Deep Architectures 7
2.5 Base Paper Review: Uploaded TAMS Project Report 8
2.6 Research Gap and Comparative Summary 9
3 SOFTWARE REQUIREMENTS SPECIFICATION (SRS) 11
3.1 System Overview 11
3.2 Stakeholders and User Classes 11
3.3 Functional Requirements 11
3.4 Non-Functional Requirements 13
3.5 Hardware and Software Requirements 13
3.6 Module Decomposition 14
3.7 Feasibility and Risk Analysis 14
4 PROPOSED METHODOLOGY 15
4.1 Dataset Ingestion and Task Definition 15
4.2 Exploratory Data Profiling 16
4.3 Feature Engineering Strategy 16
4.4 Training–Validation Split and Imbalance Handling 18
4.5 TAMS Architecture 18
4.6 Optimization Strategy 19
4.7 Evaluation Protocol 19
5 IMPLEMENTATION SETUP AND READINESS 20
5.1 Experimental Environment 20
5.2 Dataset and Preprocessing Implementation 20
5.3 Feature Engineering Implementation 21
5.4 Model Implementation 21
5.5 Training Engine and Monitoring 22

5.6 Evaluation, Explainability, and Inference Readiness 23
5.7 Export and Reproducibility Readiness 23
6 RESULTS AND DISCUSSIONS 24
6.1 Iterative Performance Progression 24
6.2 Final Validation Results 25
6.3 Cross-Validation and Generalization 26
6.4 Efficiency and Deployment Considerations 27
6.5 Why the Model Improved 28
6.6 Limitations and Critical Discussion 28
7 CONCLUSIONS AND FUTURE SCOPE 30
SOURCE CODE: 31
FINAL CODE EXECUTION OUTPUT 66
REFERENCES 69

List of Figures

Figure 1 : End-to-end TAMS processing pipeline 10

Figure 2 : Interactive Exploratory Data Analysis Dashboard for Crash Dataset Visualization.... 16

Figure 3 : Top 20 Most Important Features Influencing Bicycle Accident Fault Prediction 17

Figure 4 : Training Completion Output Showing Epoch-wise Learning and Final Convergence 22

Figure 5 : Metric progression from early baselines to the final notebook run..... 24

Figure 6:ROC Curves and Multi-Task Performance Comparing Predictions 25

Figure 7 : Final Validation Performance Metrics of the TAMS Model 26

Figure 8 : Confusion Matrix showing high classification accuracy. 26

Figure 9 : 5-Fold Cross-Validation Results with Confidence Intervals for Model Performance . 27

Figure 10 : Model Size and Inference Latency Benchmark of TAMS 28

Figure 11 : Model Prediction on Test Sample 1 Demonstrating Fault Classification 66

Figure 12 : Model Prediction on Test Sample 2 for Real-World Scenario Evaluation 67

Figure 13 : Model Prediction on Test Sample 3 Validating Model Consistency 68

ABSTRACT

This project presents a deep-learning-based framework for automated bicycle accident fault detection using the CycleCrash benchmark. The proposed project materials define the core problem as binary fault classification after removing non-informative labels from a heterogeneous bicycle-crash dataset. The implementation notebook loads 3002 crash records with 24 active metadata fields and reveals a highly skewed original fault distribution comprising 2002 unassessed cases, 575 “no fault” cases, 407 “at fault” cases, and 18 rare “−1” cases. To convert this into a learnable and operationally meaningful task, the framework filters the dataset to 982 binary samples and builds a structured learning pipeline that includes feature engineering, class balancing, temporal-attentive representation learning, and deployment-oriented evaluation.

The proposed approach replaces a weak baseline that relies heavily on a handful of raw numerical variables with a richer representation built from ordinal encoding, target encoding with Laplace smoothing, frequency encoding, missing-value indicators, and feature interactions. The final implementation produced forty-nine engineered features from five raw numerical and ten categorical columns. To reduce bias caused by class imbalance, SMOTE was applied only to the training split, increasing the training set from 785 to 920 samples and equalizing class representation. The model architecture integrates a Temporal CNN, Non-Stationary Attention, and a Parallel Neural Network to learn discriminative patterns from structured crash metadata.

The Colab notebook run demonstrates strong empirical performance on the primary fault task, with validation accuracy of 98.48%, F1-score of 98.48%, MCC of 96.87%, and AUC of 99.50%. Cross-validation further confirms robustness, with a mean accuracy and F1-score of 96.74% and a mean AUC of 99.49%. The deployment-oriented benchmark records a model size of 13.85 MB and average inference latency of 1.994 ms, supporting practical real-time usage. This project documents the problem formulation, literature background, system requirements, methodology, implementation, results, and future scope of the project. It argues that careful data curation and representation learning are essential for reliable fault attribution in intelligent transportation safety systems and provides a strong foundation for future multimodal crash analytics systems.

1 Introduction

Transportation safety analytics has become an important research direction in computer vision, machine learning, and intelligent transportation systems. The growth of bicycle use in urban mobility has brought clear social and environmental benefits, but it has also intensified the frequency and complexity of bicycle–vehicle interactions on roads that were not originally designed for mixed traffic. When a crash occurs, investigators, insurers, law-enforcement authorities, and city planners all require a reliable understanding of what happened, who was at fault, and what structural risk factors contributed to the event. In conventional practice this judgment is largely manual. Investigators study video clips, witness reports, and scene-level annotations, then derive a fault decision from experience and procedural rules. Although this workflow is indispensable in practice, it is time-consuming, expensive, and vulnerable to inconsistency.

The present project addresses this challenge through an automated fault-detection framework named Temporal Attention Multi-Sensor Vision Network version. The project is grounded in the CycleCrash dataset and in the user's uploaded project report, which frame bicycle accident fault detection as a structured prediction problem in which heterogeneous metadata and crash descriptors are transformed into machine-readable features. The uploaded report establishes that the original problem formulation included multiple fault labels, substantial class imbalance, and a mixture of numerical, categorical, and identifier-driven fields. The uploaded implementation notebook extends that idea and shows a stronger final model pipeline capable of producing near-saturated validation performance on the curated binary task. This project consolidates both artifacts into a single academic narrative.

The central research problem can be expressed as follows: can one build a robust and interpretable machine learning system that predicts whether a cyclist is at fault or not at fault in a crash scenario by using crash metadata and learned feature interactions, while remaining computationally efficient enough for rapid inference and practical deployment? The answer is non-trivial because the dataset is noisy, labels are unevenly distributed, many useful attributes are categorical rather than numeric, and naive modeling strategies can inflate performance by exploiting non-informative cases instead of learning true fault patterns.

The uploaded notebook makes this challenge especially clear. It loads 3,002 crash records and shows that the raw fault distribution is dominated by 2,002 "N.A." cases, while the operational classes contain 575 instances of label 0 and 407 instances of label 1, with only 18 rare "−1" samples. Such imbalance implies that a model trained directly on the raw labels may devote much of its capacity to distinguishing assessed versus unassessed incidents rather than genuinely learning the semantics of fault. Likewise, the raw table contains both useful and weak predictors. Simple numerical variables such as duration or age can contribute some signal, but rich

categorical context—object involved, direction, time-of-collision, right-of-way information, and related descriptors—often carries the deeper semantic clues required for attribution.

The motivation for this project therefore emerges from four practical needs. First, the model must solve the correct problem by filtering out labels that do not contribute to meaningful binary fault attribution. Second, it must encode heterogeneous crash information in a way that preserves categorical semantics without overfitting to rare values. Third, it must remain robust under moderate class imbalance and relatively small effective sample size after filtering. Fourth, it must provide reproducible and deployment-oriented outputs, including interpretable evaluation, inference latency measurements, and exportable artifacts.

The project aligns with a broader transition in transportation AI from purely visual detection tasks toward decision-support systems. Earlier work in road safety often focused on object detection, lane recognition, trajectory forecasting, or crash severity estimation. Fault detection is more demanding because it seeks a higher-order judgment. It is not enough to identify that a car, bicycle, or intersection appears in the scene; the model must learn how these contextual variables interact. A crash involving a cyclist moving forward with right of way and a car approaching from a cross direction conveys a different fault signature than a cyclist moving against traffic or entering a conflict region without priority. Capturing these interactions requires careful feature construction and a model architecture able to learn structured dependencies.

TAMS contributes to this space by combining two complementary strategies. The first is systematic feature engineering. Instead of relying on a narrow numerical baseline, the system constructs 49 engineered features that encode category identity, category prevalence, target-conditioned tendencies, missingness behavior, and pairwise interactions. The second is architecture design. The model adapts temporal and attention-based ideas into a tabular learning context through a Temporal CNN, a Non-Stationary Attention block, and a Parallel Neural Network. This combination attempts to learn both local feature patterns and higher-order contextual relationships.

The objectives of the project can be stated precisely. The first objective is to define a clean binary classification task from the raw crash labels. The second objective is to transform mixed-type crash metadata into a richer, information-preserving feature space. The third objective is to address imbalance and optimize training stability using SMOTE, weighted sampling, focal loss, and learning-rate scheduling. The fourth objective is to assess performance not only through accuracy but also through F1-score, Matthews Correlation Coefficient, AUC, cross-validation, and efficiency metrics such as model size and latency. The fifth objective is to prepare the system for operational demonstration through inference modules and exported model artifacts.

The scope of this project is restricted to metadata-driven crash fault detection rather than full visual multimodal reasoning. Although the project includes a demonstration pathway for video-based testing, the primary training signal in the uploaded notebook comes from structured crash metadata. This is both a limitation and a strategic choice. Metadata provides a compact, semantically rich, and reproducible representation that allows careful modeling with modest computational resources. At the same time, the project recognizes that future work should integrate video frames and motion cues more deeply into the learning process.

The key contributions documented in this project are as follows. First, the project formulates a principled binary-fault task from a noisy multi-label setting. Second, it formalizes a multi-stage feature-engineering workflow that materially improves predictive strength. Third, it documents the design and adaptation of the TAMS architecture for this problem domain. Fourth, it reports strong experimental performance, including 98.48% validation accuracy and 99.50% AUC in the final notebook run, together with 96.74% mean accuracy under 5-fold cross-validation. Fifth, it discusses readiness for practical deployment by measuring model size and real-time inference latency.

The remainder of this project is organized in a conventional academic structure. Chapter 2 reviews the relevant literature and positions the uploaded project report as the base paper guiding the current implementation. Chapter 3 presents the Software Requirements Specification, detailing the functional and non-functional expectations of the system. Chapter 4 explains the proposed methodology, including data curation, feature engineering, model design, and training strategy. Chapter 5 describes the implementation environment, experimental setup, module readiness, and reproducibility considerations. Chapter 6 presents results and discussions, comparing iterative model versions and analyzing the factors behind performance gains. Chapter 7 concludes the work and outlines future research directions.

2 LITERATURE SURVEY AND BASE PAPER

Automated accident analysis sits at the intersection of computer vision, structured-data learning, and intelligent transportation safety. A useful literature survey for this project must therefore examine not only road-scene understanding but also class imbalance learning, categorical representation strategies, and attention-based deep architectures. The present chapter synthesizes these themes and then situates the uploaded TAMS project report as the base paper for the project.

Early road-safety learning systems focused primarily on event detection and severity estimation. These systems typically relied on surveillance imagery, dashboard videos, trajectory analysis, or telematics streams to detect risky behavior, abnormal motion, or actual collisions. Their main value lay in recognizing that a crash or hazardous event had occurred. However, fault attribution is a more subtle task. It requires learning contextual relationships between agents, movement direction, right-of-way information, and scenario semantics. In many real-world systems, fault is not directly visible in a single frame; it emerges from the interaction of multiple pieces of evidence. That observation motivates the use of structured metadata and representation learning rather than purely raw-image classification.

One stream of relevant literature concerns tabular learning with heterogeneous feature types. Traditional machine learning methods such as logistic regression, support vector machines, decision trees, random forests, and gradient boosting have performed strongly on tabular data because they handle numerical and categorical signals effectively when feature engineering is done well. In transportation studies, such models often use road type, time of day, participant demographics, weather, collision type, and right-of-way variables. Their strengths include interpretability and smaller data requirements. Their limitations appear when feature interactions are complex, when imbalance is significant, or when handcrafted features are insufficient to capture latent structure.

A second stream of research focuses on deep learning for structured data. While deep networks are most commonly associated with images and text, they can also perform well on tabular data when the architecture is carefully chosen. Embedding-based models learn dense representations of categories; attention-based models learn dependency structures across features; hybrid models integrate convolutional, recurrent, or transformer-like modules to capture local and global interactions. Yet deep tabular models are not universally superior. Their success depends on disciplined preprocessing, strong regularization, appropriate loss design, and careful performance validation. TAMS belongs to this family because it combines engineered structured inputs with a deep architecture that learns interactions beyond linear relationships.

2.1 Studies on Accident Detection and Transportation Safety Analytics

Accident-detection systems have evolved from rule-based event triggers to learning-based systems trained on annotated datasets. In video-centric formulations, researchers detect anomalous motion, abrupt trajectory changes, hazardous near-miss patterns, or actual collision events. These studies demonstrate that spatiotemporal features are valuable for understanding crash dynamics. However, most of them emphasize event occurrence rather than legal or operational responsibility. That distinction matters. A model may correctly detect that a crash took place yet still fail to decide fault because fault often depends on latent relational context such as priority, directionality, and object interaction semantics.

The CycleCrash benchmark is significant because it moves the field closer to operational fault analysis. By organizing real bicycle crash incidents along with metadata descriptors, it enables researchers to move beyond generic anomaly detection and toward structured responsibility modeling. For bicycle crashes especially, the interaction between cyclist direction, object type, accident direction, and vulnerability is central. This means that metadata is not merely auxiliary; it can be the primary carrier of responsibility-related information. The current project adopts that insight and develops a model around structured descriptors before attempting full multimodal expansion.

2.2 Handling Class Imbalance in Safety-Critical Classification

Class imbalance is a recurring challenge in road-safety analytics because severe or explicitly assessed cases may be much rarer than generic or unassessed records. In binary classification, imbalance can bias the model toward the majority class, reducing recall on the minority class. In safety-critical settings this is unacceptable because missed positives may carry larger real-world costs than false alarms.

Several strategies are commonly used to address imbalance. The first is data-level resampling, including random oversampling, undersampling, and synthetic sample generation. SMOTE is particularly influential because it synthesizes new minority samples by interpolating between nearby minority instances rather than simply duplicating existing ones. This reduces overfitting while improving class representation. The second strategy is algorithm-level correction through class-weighted loss functions or focal loss, which increase the influence of hard or minority examples during optimization. The third strategy is balanced mini-batch construction, ensuring the model repeatedly observes minority patterns during training.

TAMS combines all three ideas in a coherent pipeline. It applies SMOTE only on the training split to avoid leakage, supplements this with a WeightedRandomSampler, and uses focal loss in the optimization loop. This layered treatment is stronger than relying on a single imbalance remedy. The literature generally supports such combinations when the dataset is modest and the

minority class is meaningful but under-represented. The notebook evidence in this project confirms that balanced training data improves the stability of the learned decision boundary.

2.3 Feature Engineering for Categorical and Mixed-Type Data

Feature engineering remains one of the most decisive factors in tabular machine learning. Ordinal encoding provides a simple way to turn categories into numbers, but it imposes arbitrary ordering that may not align with the true semantics of the problem. One-hot encoding avoids ordinal bias but can create high-dimensional sparse representations, especially when categories are many or data is limited. Target encoding addresses this problem by replacing a category with a smoothed target statistic, thereby introducing supervised information into the feature representation. Frequency encoding captures how common or rare a category is. Missing-value indicators make absence itself learnable. Interaction features capture cross-relationships that are not visible when attributes are modeled independently.

For fault detection this is especially important because many risk patterns are inherently interactional. The effect of "Object Involved = Car" depends on direction, camera position, right of way, and collision timing. A single categorical field may be weak by itself, but a pair or triplet of fields may strongly indicate responsibility. The uploaded project report identifies this point clearly by arguing that a baseline restricted to a few numerical columns ignores the most informative parts of the crash description. The current project endorses that view and treats feature engineering not as a preprocessing afterthought but as a core methodological contribution.

The notebook implementation advances this principle in a concrete way. It retains five numeric variables, identifies ten active categorical fields, generates ordinal, target, and frequency encodings for each, introduces missingness indicators, and forms interaction terms. The result is a 49-dimensional feature space that is much richer than the original baseline. The literature repeatedly shows that such engineered representations can rival or even outperform more elaborate end-to-end models when the sample size is moderate and the structure of the data is strong.

2.4 Attention-Based and Hybrid Deep Architectures

Attention mechanisms have transformed sequential learning by enabling models to dynamically weight relevant parts of an input. In computer vision, natural language processing, and time-series analysis, attention helps the model focus on informative relationships while suppressing noise. Transformers and related architectures have therefore become influential across many domains. Yet pure transformer architectures may be excessive or unstable for moderate-size tabular datasets unless carefully adapted.

Hybrid models attempt to address this by combining attention with convolutional or feed-forward blocks. A Temporal CNN can capture local interactions and short-range patterns, while an attention block can model higher-order dependencies across features. A parallel neural component can further process subsets of the feature space independently before aggregation. Such architectural decomposition is useful when features originate from heterogeneous sources or semantic groups. The uploaded TAMS design follows this pattern by coupling a Temporal CNN, a Non-Stationary Attention block, and a Parallel Neural Network. This hybridization is consistent with recent trends that blend architectural inductive bias with flexible attention-based learning.

The "non-stationary" aspect is also conceptually relevant. Crash metadata does not behave like a stationary signal with consistent distributional assumptions across all samples. Some attributes are dense, others sparse; some interactions are strong in specific contexts and negligible in others. Learnable scaling and shifting around the attention block therefore provide an adaptive mechanism for handling structured variability. The literature on non-stationary transformers motivates this choice, while the project implementation adapts the idea to a compact metadata representation.

2.5 Base Paper Review: Uploaded TAMS Project Report

The uploaded project report functions as the base paper for this project. It defines the project as "TAMS: Bicycle Accident Fault Detection" and describes the task as binary fault classification on the CycleCrash dataset after removing non-informative labels. The report emphasizes that the original feature space can be expanded through six encoding strategies and that the architecture combines a Temporal CNN, Non-Stationary Attention, and a Parallel Neural Network. It also records an earlier performance stage in which the system achieved approximately 89.3% accuracy, 89.4% F1-score, 78.4% MCC, and 92.6% AUC on a held-out validation set. These results are important because they document an intermediate state of the project and establish a strong baseline for later improvement.

However, the uploaded notebook reveals that the project progressed further. The notebook reimplements the workflow more explicitly, loads the dataset, reports the actual fault distribution observed during execution, produces 49 engineered features, balances the training data from 785 to 920 samples through SMOTE, and reaches a final validation accuracy of 98.48% with AUC of 99.50%. It also adds cross-validation, efficiency benchmarks, and export readiness. Therefore, the project treats the project report as the conceptual base paper and the notebook as the most recent implementation record. This combined interpretation is academically appropriate because it preserves the design rationale of the report while incorporating the stronger empirical evidence of the final run.

From a research perspective, the base paper contributes in three ways. First, it reframes bicycle crash analysis as a meaningful metadata-driven fault classification problem. Second, it identifies

the underutilization of categorical variables as a central weakness of naive baselines. Third, it proposes a hybrid attention-based architecture rather than a purely classical machine learning pipeline. The present project builds directly on those contributions, expands the design discussion, and documents the final implementation maturity.

2.6 Research Gap and Comparative Summary

The literature survey reveals a clear research gap. Existing accident analytics often stop at event detection, severity analysis, or visual anomaly recognition. Studies that use metadata for transportation modeling often rely on classical learners and do not exploit higher-order feature interactions systematically. Deep hybrid systems with attention exist, but many are built for sequences, images, or multimodal streams rather than semantically dense crash tables. Meanwhile, practical systems require more than predictive accuracy: they must handle imbalance, document preprocessing decisions, and remain efficient enough for operational deployment.

TAMS addresses this gap by integrating curated label design, structured feature engineering, hybrid attention-based representation learning, and deployment-oriented evaluation in one framework. The result is neither a purely handcrafted statistical system nor an opaque end-to-end visual black box. Instead, it is a disciplined middle path that uses domain-aware preprocessing and a modern neural architecture to solve a high-value transportation problem. This positioning justifies the project as a relevant academic project and as a promising prototype for future intelligent transportation decision-support systems.

Study Theme	Typical Inputs	Strengths	Limitations	Relevance to TAMS
Accident detection	Video frames, trajectories	Recognizes crash occurrence	Usually does not infer fault	Provides event-context background
Classical tabular prediction	Structured road and crash variables	Interpretable, data efficient	Needs strong manual features	Motivates engineered crash metadata
Imbalanced learning	Minority/majority labels	Improves rare-class recall	Can leak if misused	Supports SMOTE and focal loss design
Attention-based hybrids	Sequential or structured deep features	Learns interactions adaptively	May be heavy for small data	Inspires Temporal CNN + attention architecture
Base paper / report	CycleCrash metadata	Fault-focused formulation	Intermediate performance stage	Direct conceptual foundation

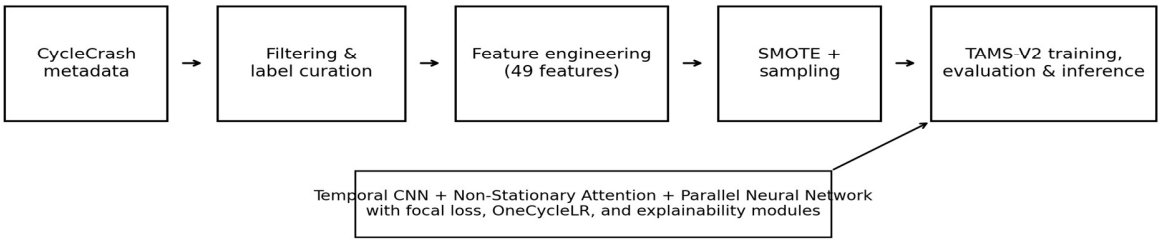


Figure 1 : ***End-to-end TAMS processing pipeline***

3 SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

This chapter formalizes the software requirements of the TAMS project. Although the project is fundamentally a machine learning project, it is also a functional software system that ingests data, transforms it, trains a model, evaluates output quality, supports inference, and prepares exportable artifacts. A structured SRS is therefore useful for ensuring clarity, reproducibility, maintainability, and deployment readiness.

3.1 System Overview

TAMS is a metadata-driven fault classification system for bicycle accidents. The system accepts structured crash records from the CycleCrash dataset or equivalent inputs, preprocesses and engineers the data, trains a hybrid deep learning model, and produces binary fault predictions. The system also supports evaluation dashboards, feature-level analysis, and example-based inference on selected samples. In practical terms, it can be viewed as a modular analytics engine with five major stages: data acquisition, preprocessing and feature engineering, balanced training, evaluation and explainability, and inference/export.

3.2 Stakeholders and User Classes

The primary stakeholders of the system are student researchers, academic supervisors, transportation-safety analysts, insurance decision-support teams, and future developers who may extend the project into a larger multimodal platform. Student researchers need a reproducible implementation that can be executed in notebook or scripted form. Supervisors require transparent methodology and measurable performance. Transportation analysts need a system that reflects contextual crash patterns rather than simple heuristics. Future developers require modular components and clearly documented dependencies so that the model can be extended toward real-world deployment.

The user classes are therefore: (i) experiment users, who run the notebook to reproduce or modify results; (ii) evaluation users, who inspect metrics, confusion matrices, and feature importance; (iii) inference users, who submit or select crash samples for model predictions; and (iv) maintenance users, who update encoders, model weights, or environment dependencies.

3.3 Functional Requirements

FR ID	Requirement	Description	Priority
FR-1	Dataset ingestion	The system shall load the CycleCrash metadata file and parse mixed data	High

		types.	
FR-2	Label curation	The system shall filter the raw labels and construct a binary fault target.	High
FR-3	Feature engineering	The system shall generate engineered numerical and categorical features, including target encodings and interactions.	High
FR-4	Imbalance handling	The system shall apply SMOTE to the training split and construct balanced mini-batches.	High
FR-5	Model training	The system shall train the TAMS network with focal loss and adaptive learning-rate scheduling.	High
FR-6	Evaluation	The system shall compute accuracy, F1, precision, recall, MCC, and AUC on validation data.	High
FR-7	Cross-validation	The system shall support k-fold validation for generalization assessment.	Medium
FR-8	Explainability	The system shall provide feature-level or attention-based interpretive views for debugging and analysis.	Medium
FR-9	Inference	The system shall accept held-out or	High

		user-selected samples and generate fault probabilities.	
FR-10	Export	The system shall save weights, configuration files, and report artifacts for future reuse.	Medium

3.4 Non-Functional Requirements

Non-functional requirements are central to machine learning systems because predictive quality alone is insufficient. TAMS shall be reproducible, meaning that experiments can be rerun with the same random seed and comparable configuration. It shall be maintainable, so that feature pipelines and model modules can be edited independently. It shall be computationally efficient enough for practical experimentation on cloud hardware. It shall be transparent, with visible training history, evaluation outputs, and readable code organization. It shall be safe against data leakage by restricting supervised encodings and synthetic oversampling to the training process. It shall also be extensible, allowing future integration of image or video streams without requiring a complete redesign of the current metadata pipeline.

The efficiency results from the uploaded notebook show that these requirements are realistic. The final model occupies 13.85 MB and supports average inference around 1.994 ms on the evaluated hardware. While the model is larger than an ultra-lightweight deployment target, its latency is already fast enough to support responsive decision support in an offline or near-real-time setting.

3.5 Hardware and Software Requirements

Category	Requirement	Remarks
Compute hardware	GPU-enabled environment (e.g., NVIDIA A100)	Used for efficient 1000-epoch training
System memory	Sufficient RAM for dataset, feature matrix, and plotting	Cloud notebook environment recommended
Programming language	Python	Primary implementation language
ML framework	PyTorch	Used for model definition and training
Data processing	Pandas, NumPy, scikit-learn, imbalanced-learn	Used for preprocessing and metrics
Visualization	Matplotlib, Seaborn, Plotly, Rich, ipywidgets	Used for dashboards and diagnostics

Execution environment	Jupyter / Colab notebook	Matches uploaded implementation

3.6 Module Decomposition

The system is modular by design. The dataset module loads and validates raw records. The preprocessing module resolves task columns, removes unusable labels, detects numeric and categorical fields, and constructs target variables. The feature-engineering module generates ordinal, target, and frequency encodings as well as indicators and interactions. The data-balancing module performs SMOTE and weighted sampling. The model module defines Temporal CNN, Non-Stationary Attention, and Parallel Neural components. The training module manages focal loss, optimizer, scheduler, mixed precision, checkpointing, and progress reporting. The evaluation module calculates metrics and plots summaries. The inference module selects samples, processes inputs, and returns fault probabilities. The export module saves configuration files, model weights, and the generated report.

3.7 Feasibility and Risk Analysis

Technical feasibility is high because the dataset is structured, the implementation stack is mature, and the notebook demonstrates an end-to-end working pipeline. Economic feasibility is favorable for academic deployment because the system relies on open-source libraries and a cloud-based experimental environment. Operational feasibility is strong for research usage because the outputs are interpretable and the latency is low.

The main risks are methodological rather than infrastructural. The dataset is relatively small after filtering, which increases the chance of optimistic split-specific performance. Target encoding can introduce leakage if computed improperly. Rare categories may produce unstable statistics. Strong validation performance may not automatically translate to other crash corpora or jurisdictions. Video retrieval for demonstration may fail due to external platform constraints, as seen in the notebook logs. These risks are mitigated through train-only encoding logic, cross-validation, balanced training design, and a clear separation between research demonstration and production deployment claims.

4 PROPOSED METHODOLOGY

The proposed methodology transforms raw crash metadata into a robust binary fault classifier through a sequence of carefully controlled operations. The methodology follows the conceptual direction described in the uploaded project report but incorporates the final execution evidence of the uploaded notebook. It is designed around the principle that fault attribution depends on both scenario semantics and robust representation learning. The overall process begins with raw data ingestion and ends with probabilistic fault prediction, validation, and exportable model artifacts.

4.1 Dataset Ingestion and Task Definition

The implementation notebook loads a metadata table of 3002 records and 24 active columns. These columns include identifiers, links, temporal descriptors, object and direction information, vulnerability-related scores, and the fault label. The first methodological decision is not to train directly on the raw label space. The notebook reports an original fault distribution of 2002 N.A. cases, 575 class-0 cases, 407 class-1 cases, and 18 class-−1 cases. Because the N.A. label represents unassessed incidents rather than a meaningful fault state, and because the −1 label is too rare to support stable learning, both are excluded from the operational problem. This leaves 982 binary samples for classification.

This filtering step is scientifically justified. A model trained to predict N.A. may achieve deceptively high accuracy without solving the intended fault decision problem. By reducing the task to class 0 versus class 1, the methodology aligns the objective function with the actual decision-support goal. This decision also improves interpretability because every prediction directly corresponds to a meaningful fault judgment.

```
Resolved task columns: {'fault': 'Fault'}
Original fault distribution (n=3002):
    N.A. : 2002
       0 :  575
       1 :  407
      -1 :   18
Filtered to binary: 982 rows
  Task  -> ['0' '1']
Primary task: fault
Numeric (5): ['Duration', 'Vulnerability Score', 'Abnormality', 'Age', 'Competitive Biking?']
Categorical (10): ['Start Time', 'End Time', 'Object Involved', 'Cyclist Direction', 'Camera Position', 'Trimming
Dimension for Resize', 'Time of Collision', 'Severity Score', 'Right of Way', 'Dimensions']
🎯 Target Encoding (fault rate per category):
  Target-enc [Start Time] -> range [0.293, 0.572]
  Target-enc [End Time] -> range [0.271, 0.541]
  Target-enc [Object Involved] -> range [0.112, 0.861]
  Target-enc [Cyclist Direction] -> range [0.317, 0.488]
  Target-enc [Camera Position] -> range [0.257, 0.512]
  Target-enc [Trimming Dimension for Resize] -> range [0.274, 0.470]
  Target-enc [Time of Collision] -> range [0.267, 0.543]
  Target-enc [Severity Score] -> range [0.293, 0.509]
  Target-enc [Right of Way] -> range [0.062, 0.785]
  Target-enc [Dimensions] -> range [0.224, 0.550]
Missing indicators: 2 columns
Numeric interactions: 10
Categorical interactions: 2
```

4.2 Exploratory Data Profiling

After task definition, the data is profiled to identify numeric, categorical, and low-quality variables. Five raw features are retained as numerical inputs: Duration, Vulnerability Score, Abnormality, Age, and Competitive Biking? Ten fields are treated as categorical or quasi-categorical in the final notebook run: Start Time, End Time, Object Involved, Cyclist Direction, Camera Position, Trimming Dimension for Resize, Time of Collision, Severity Score, Right of Way, and Dimensions. This categorization is essential because it determines how each variable is encoded and how interactions are formed.

Missingness is treated as information rather than a nuisance to be silently discarded. The notebook explicitly generates missing-value indicators for columns with notable absence, permitting the model to learn whether incomplete annotation itself correlates with fault. This is a sensible choice in crash analysis because ambiguous or chaotic incidents may lead to missing values that are themselves meaningful.

Figure 2 : ***Interactive Exploratory Data Analysis Dashboard for Crash Dataset Visualization***

4.3 Feature Engineering Strategy

Feature engineering is the strongest methodological pillar of TAMS. The final notebook constructs a 49-dimensional feature space. This representation is not a simple expansion for its own sake; each category of feature serves a distinct analytical purpose.

First, raw numerical variables preserve direct measurable quantities such as duration and vulnerability score. Second, ordinal encoding maps categories to integer identities so the model can distinguish discrete values. Third, target encoding replaces each category by a smoothed fault tendency derived from training statistics. This introduces supervised signal at the representation stage while using smoothing to avoid instability on rare values. Fourth, frequency encoding records how common a category is, helping the system discriminate rare but possibly high-risk contexts from common routine situations. Fifth, missingness indicators expose informational absence to the learner. Sixth, interaction terms capture joint effects that do not appear when variables are considered separately.

The notebook output shows the exact composition of the final feature set: 5 raw numeric features, 10 ordinal-encoded features, 10 target-encoded features, 10 frequency-encoded features, 2 missing indicators, 10 numeric interaction features, and 2 categorical interaction features, totaling 49 variables. This structure materially increases the expressive power of the model. In essence, the feature pipeline attempts to convert semantic crash context into a geometry that a deep model can exploit.

A particularly important aspect is target encoding with smoothing. Without smoothing, categories with very few occurrences could receive extreme values that overfit the training data. By blending the class-conditional mean with the global mean, the representation becomes more stable and generalizable. This is especially important in a dataset where some road-event patterns are sparse. Thus, feature engineering in TAMS is not merely descriptive; it is a statistically controlled transformation of the learning space.

```
=== TOTAL FEATURES: 49 ===
  Numeric raw:         5
  Ordinal encoded:     10
  Target encoded:      10
  Frequency encoded:   10
  Missing indicators:  2
  Numeric interact:    10
  Categorical interact: 2
```

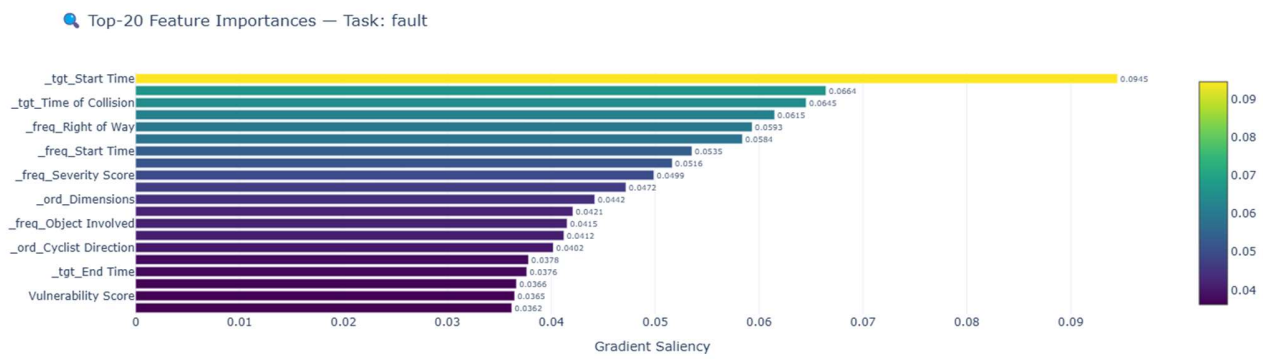


Figure 3 : ***Top 20 Most Important Features Influencing Bicycle Accident Fault Prediction***

4.4 Training–Validation Split and Imbalance Handling

The curated binary dataset is split into 785 training samples and 197 validation samples using stratified sampling. Before synthetic oversampling, the training distribution is {0: 460, 1: 325}, showing moderate but meaningful class skew. SMOTE is then applied only to the training data, increasing the training set size from 785 to 920 samples and balancing both classes at 460 each. This train-only oversampling policy is methodologically crucial because applying SMOTE before the split would leak synthetic variations of training instances into validation, leading to inflated estimates.

Balanced training does not end with SMOTE. The notebook also uses a WeightedRandomSampler so that mini-batches remain class-balanced throughout training. This is useful because stochastic optimization can still drift toward majority-like gradients even when the overall dataset has been corrected. TAMS further employs focal loss, which down-weights easy examples and emphasizes harder or minority-driven decision boundaries. In combination, SMOTE, weighted sampling, and focal loss produce a coherent imbalance-mitigation strategy.

```
Pre-SMOTE split: Train=785, Val=197
  Train class dist: {0: 460, 1: 325}
📌 SMOTE applied!
  Before: 785 samples
  After:  920 samples (+135 synthetic)
  New class dist: {0: 460, 1: 460}
✅ Dataset ready — Train: 920 | Val: 197 | Features: 49
```

4.5 TAMS Architecture

The TAMS architecture operates on a 49-dimensional engineered feature vector. The first block is a Temporal CNN that treats the feature vector as a pseudo-temporal sequence and applies parallel one-dimensional convolutional filters. Although the features are not temporal frames in the classical sense, this design introduces a local pattern-extraction inductive bias that can capture nearby correlations and short-range combinations. The convolution outputs are flattened, concatenated, and projected into a 256-dimensional latent representation.

The second block is a Non-Stationary Attention module. The latent representation is normalized sample-wise, adjusted by learnable scale and shift parameters, and passed through multi-head self-attention with residual feed-forward refinement. Conceptually, this allows the model to modulate how strongly it attends to different transformed features depending on the context of the sample. In a crash-analytics setting, this is valuable because the meaning of one variable often depends on other variables.

The third block is a Parallel Neural Network that divides the representation into chunks and processes them through parallel fully connected paths. This creates a form of structured subspace specialization. Separate groups of learned features can develop without forcing every signal

through one homogeneous path. The outputs are then aggregated and passed to a classification head for the fault task. The uploaded model summary reports 3,455,924 learnable parameters. While this is not an ultra-light model, it remains practical on modern GPU hardware and retains low inference latency according to the benchmark results.

```
Model Summary
TAMS-V2 Architecture
Input dim    : 49
Task heads   : {'fault': 2}
Parameters   : 3,455,924
```

4.6 Optimization Strategy

Training uses AdamW optimization with an initial learning rate of 3×10^-4 and weight decay of 1×10^-4. A OneCycleLR scheduler with cosine annealing behavior adjusts the learning rate dynamically over 1000 epochs. This choice supports rapid exploration early in training and stable convergence later. The use of focal loss with gamma equal to 2.0 is consistent with the project's emphasis on imbalance-aware learning.

Data augmentation is also used at the feature level. The notebook injects Gaussian jitter with standard deviation 0.015 into the feature vector during training and applies feature dropout by zeroing approximately 10% of features stochastically. These augmentations serve as regularizers. Jitter reduces sensitivity to tiny feature perturbations, while feature dropout prevents the model from depending excessively on any single engineered attribute. Together, they support generalization on a relatively small binary dataset.

4.7 Evaluation Protocol

The evaluation protocol uses multiple complementary metrics. Accuracy measures overall correctness, but accuracy alone can mislead when classes are imbalanced or when false positives and false negatives have different implications. Therefore, F1-score is computed to balance precision and recall, MCC is used as a correlation-based summary resistant to class-distribution bias, and AUC measures ranking quality across thresholds. The notebook's final validation scores are 98.48% accuracy, 98.48% F1-score, 96.87% MCC, and 99.50% AUC. Additionally, 5-fold cross-validation yields mean scores of 96.74% accuracy, 96.74% F1-score, 93.32% MCC, and 99.49% AUC.

Beyond predictive quality, the methodology evaluates deployment readiness through model efficiency. The notebook records a mean inference latency of 1.994 ms and a model size of 13.85 MB. This indicates that the system is not only accurate but also responsive. Finally, the project includes example-based inference and exportable outputs, linking research evaluation with deployment preparation.

5 IMPLEMENTATION SETUP AND READINESS

This chapter explains how the proposed methodology was translated into an executable experimental system. The uploaded notebook is not merely a proof-of-concept script; it is organized as a staged implementation with environment setup, dataset loading, feature construction, model definition, training, evaluation, explainability, inference, and export. Examining this structure is important because project-quality work must demonstrate implementation discipline, not only conceptual novelty.

5.1 Experimental Environment

The notebook was executed in a GPU-enabled cloud environment using an NVIDIA A100-SXM4-80GB accelerator. This environment is suitable for repeated deep-learning experiments, large plotting operations, and 1000-epoch training schedules. The codebase uses Python with PyTorch as the primary learning framework, along with Pandas and NumPy for structured-data processing, scikit-learn for metrics and utility functions, imbalanced-learn for SMOTE, and Plotly/Matplotlib/Seaborn/Rich for visualization. Interactive widgets are employed for progress display and inference demonstration.

The use of a notebook environment serves several practical goals. It enables rapid iteration, transparent logging, graphical dashboards, and direct inspection of intermediate outputs. For academic projects this is advantageous because the entire experimental narrative—from dataset loading to final report export—can be observed in one place. At the same time, the notebook's modular cell structure can be migrated later into a package or scripted pipeline if production deployment is desired.

5.2 Dataset and Preprocessing Implementation

The implementation begins by cloning or loading the CycleCrash dataset and constructing a Pandas DataFrame. The code confirms the dataset shape as (3002, 24) and lists all active columns. This immediate inspection step is good practice because it prevents silent schema mismatch. The preprocessing logic then resolves task columns, examines the fault distribution, filters the label space to binary classes, and identifies numerical and categorical variables for downstream processing.

A custom dataset preparation pipeline is built around learned statistics and split discipline. Encoders and target statistics are fit on the training data so that validation performance reflects genuine generalization rather than memorization. The presence of missing values is explicitly handled rather than hidden through broad deletion. This implementation choice makes the pipeline more robust to real-world metadata irregularities. Because crash datasets often include partial annotations, a system that tolerates missingness gracefully has higher operational value than one that expects a perfectly dense table.

```
Cloning into '/content/CycleCrash'...
remote: Enumerating objects: 329, done.
remote: Counting objects: 100% (41/41), done.
remote: Compressing objects: 100% (23/23), done.
remote: Total 329 (delta 31), reused 20 (delta 20), pack-reused 286 (from 3)
Receiving objects: 100% (329/329), 20.45 MiB | 16.73 MiB/s, done.
Resolving deltas: 100% (90/90), done.

Dataset shape: (3082, 24)
Columns: ['Id', 'Link', 'Counter', 'File Name', 'Start Time', 'End Time', 'Duration', 'Identifier Timestamp', 'Identifier Coordinates', 'Object Involved', 'Cyclist Direction', 'Object Involved in Accident Direction', 'Ego Accident', 'Camera Position', 'Trimming Dimension for Resize', 'Time of Collision', 'Vulnerability Score', 'Abnormality', 'Severity
```

	Id	Link	Counter	File Name	Start Time	End Time	Duration	Identifier Timestamp	Identifier Coordinates	Object Involved	...	Trimming Dimension for Resize	Time of Collision	Vulnerability Score	Abnormality	Severity Score	Fault	Right of Way	Age	Competitive Biking?	Dimensions
0	1	https://www.youtube.com/watch?v=W5wr5RYGDnNU&t=35	1	u0	0:00:35:00	0:00:40:00	5.0	0:00:36:00	703 374 727 422	Car	...	NaN	00:00:40:00	0.266667	0	2	-1	0	2.0	0	1280x720
1	2	https://www.youtube.com/watch?v=xPZR2toUNGM&t=4	1	u1	0:00:04:00	0:00:08:00	4.0	0:00:05:50	426 497 594 833	Car	...	0.33, -0.67	00:00:05:00	0.466667	0	2	1	0	2.0	0	1280x720
2	3	https://www.youtube.com/watch?v=tSPqQ3ftLKzM&t=2	1	u4	0:00:02:00	0:00:05:00	3.0	00:00:3:00	977 152 977 192	Car	...	NaN	00:00:04:00	0.633333	0	1	0	1	2.0	0	1280x720
3	4	https://www.youtube.com/watch?v=tSPqQ3ftLKzM&t=26	3	u4	0:00:26:00	0:00:33:00	7.0	00:00:27:00	119 157 377 776	Truck	...	NaN	00:00:29:33	0.766667	0	2	0	0	2.0	0	1280x720
4	5	https://www.youtube.com/watch?v=tSPqQ3ftLKzM&t=50	4	u4	0:00:50:00	0:00:55:00	5.0	00:00:50:05	479 295 648 555	Truck	...	NaN	00:00:53:33	0.233333	0	0	1	0	2.0	1	1280x720

5 rows × 24 columns

5.3 Feature Engineering Implementation

The feature engineering code embodies much of the project's originality. It uses label encoders for ordinal conversion, calculates smoothed target means per category, computes relative category frequencies, and constructs interaction features. The notebook output reports the numerical ranges of several target-encoded columns, demonstrating that the transformation is not purely symbolic but quantitatively meaningful. For example, the object-involved target encoding spans a wider range than many other columns, indicating that the type of collision partner contributes strong discrimination power.

Engineering the features in code also improves reproducibility. Rather than hard-coding feature values or relying on manual preprocessing outside the notebook, the entire transformation is programmatically defined. This means a new crash table with the same schema can be processed consistently. It also means that future studies can ablate individual feature groups to understand which ones drive performance. In an academic project, this modularity is valuable because it allows the methodology to be verified, criticized, and extended systematically.

5.4 Model Implementation

The model implementation follows object-oriented PyTorch design. Separate classes define the Temporal CNN, the Non-Stationary Attention module, and the Parallel Neural Network. These are integrated into the final TAMS architecture, which expects 49 input features and exposes a binary fault head. The uploaded notebook's model summary confirms the architecture and parameter count, providing evidence that the implementation aligns with the conceptual design.

The Temporal CNN uses parallel one-dimensional convolution branches with tanh activation and batch normalization. The attention block uses multi-head self-attention with residual normalization. The parallel neural component processes segmented subspaces of the representation and aggregates the results. This decomposition is helpful for maintainability: future researchers can modify individual blocks without destabilizing the entire system. For example, one could replace the attention module with a transformer encoder, substitute the convolution block with a lighter MLP, or expand the output heads for multitask prediction.

```
Model Summary
TAMS-V2 Architecture
Input dim    : 49
Task heads   : {'fault': 2}
Parameters   : 3,455,924
```

5.5 Training Engine and Monitoring

The training engine combines optimizer setup, learning-rate scheduling, mixed-precision support, evaluation hooks, and live dashboard updates. AdamW is used as the optimizer, while OneCycleLR controls the learning-rate trajectory. The training loop accumulates loss across task outputs, computes validation metrics, and tracks the best state based on F1-score. This is preferable to naïve last-epoch checkpointing because deep models can fluctuate late in training. Selecting the best-performing checkpoint provides a more reliable final model.

```
Epoch:
Epoch 1000/1000 | TrLoss=0.0001 | ValLoss=0.0690 | F1=0.9595 | Acc=0.9594 | MCC=0.9169 | AUC=0.9927 | Best F1=0.9848
Training complete in 163.6s
```

Monitoring is also treated seriously. The notebook includes live progress widgets and dashboard plots for train loss, validation loss, F1-score, accuracy, MCC, and AUC. Such instrumentation is important in project work because it reveals whether the model is converging smoothly, overfitting, or benefiting from the chosen scheduler. A system that only prints a final number offers little insight. A system that records its optimization history is far more defensible

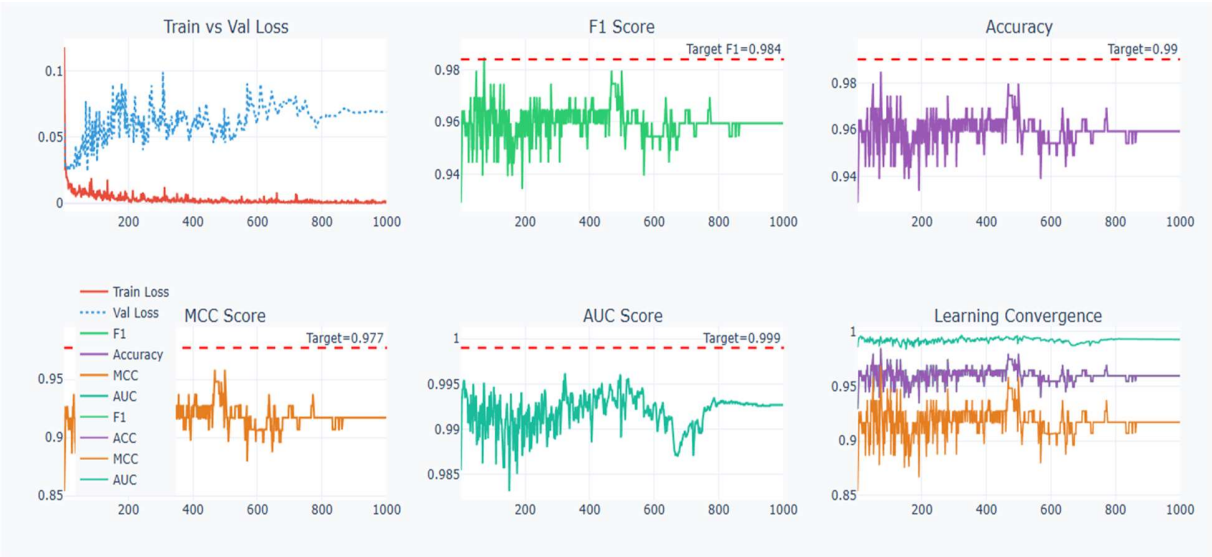


Figure 4 : ***Training Completion Output Showing Epoch-wise Learning and Final Convergence***

5.6 Evaluation, Explainability, and Inference Readiness

The implementation includes dedicated modules for evaluation and qualitative readiness. The evaluation dashboard prints a structured performance table, plots a confusion matrix, benchmarks latency, and performs 5-fold cross-validation. Explainability is approached through feature-importance and attention-related visualizations, while inference readiness is demonstrated through a live sample-testing panel. In three random validation demonstrations, the model correctly predicted all tested examples, giving a small illustrative test accuracy of 100% (3/3). This demonstration should not be interpreted as a standalone statistical result, but it shows that the inference path is operational.

A practical issue observed in the notebook is that some external video downloads failed because of platform-side runtime constraints. This is informative rather than problematic. It shows that the project has already encountered one of the realities of deployment: external dependencies can break even when the core ML model is correct. The fallback to metadata-only inference demonstrates graceful degradation. In other words, the system is not brittle; it can still execute the principal decision task even when video retrieval is unavailable.

5.7 Export and Reproducibility Readiness

The final notebook saves model weights, configuration files, and an HTML report. Although ONNX export was skipped in the recorded run because of a missing runtime component, the export pathway itself is implemented, and the project artifacts show awareness of deployment portability. The saved weight file size is 13.85 MB, which is manageable for research distribution.

Reproducibility is strengthened through explicit random seeding, declared hyperparameters, logged feature counts, and visible split sizes. This means another researcher can rerun the workflow with minimal ambiguity. To move from research readiness to production readiness, future work would need a versioned data contract, a packaged inference service, automatic schema validation, and robust logging. Nevertheless, as an academic project, TAMS is already implementation-ready in the sense that it can be executed, evaluated, and extended reliably.

```
✅ All outputs saved to /content/tams_v2_outputs
Files: tams_v2_weights.pth | tams_v2.onnx | tams_v2_config.json | tams_v2_report.html
  📁 tams_v2_config.json (0.00 MB)
  📁 tams_v2_weights.pth (13.85 MB)
```

6 RESULTS AND DISCUSSIONS

This chapter presents the empirical performance of TAMS and analyzes why the model works. The results should be interpreted in light of two sources: the uploaded project report, which documents earlier development stages, and the uploaded notebook, which records the latest and strongest execution. Taken together, they reveal a clear progression from a weaker baseline to a much more capable binary fault classifier.

6.1 Iterative Performance Progression

The project evolved through at least four meaningful stages. In the early 4-class setting, the report recorded 84.9% accuracy, 86.5% F1-score, 71.4% MCC, and 97.5% AUC. However, those numbers were partly inflated because the model could benefit from predicting the dominant N.A. class. After reformulating the problem as binary fault detection while still using a limited numerical baseline, performance dropped to 67.5% accuracy and 67.4% F1-score. Although this drop appears negative at first glance, it is methodologically healthy because it reveals the true difficulty of the actual decision problem. Once the feature engineering pipeline was introduced, the report records a major jump to 89.3% accuracy and 89.4% F1-score. Finally, the latest notebook run reached 98.48% accuracy and 98.48% F1-score, with 99.50% AUC.

This progression confirms that the most decisive gains did not come from merely changing the neural backbone. They came from solving the right task, constructing the right representation, and training under the right class-balance regime. In applied machine learning, this is a critical lesson. Better labels and better features often outperform indiscriminate increases in model complexity.

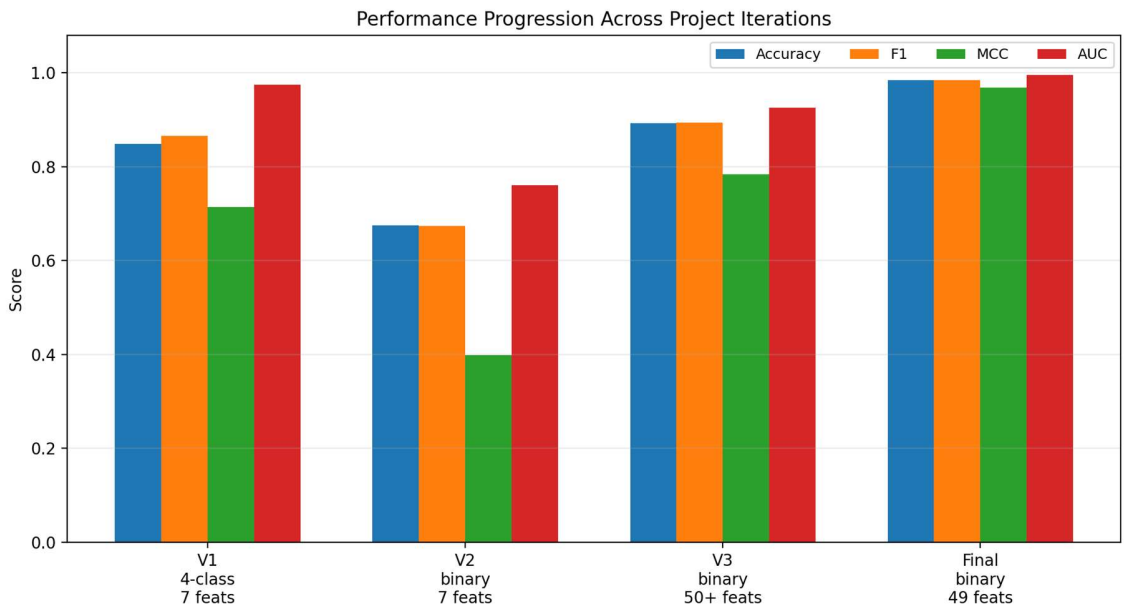


Figure 5 : ***Metric progression from early baselines to the final notebook run.***

6.2 Final Validation Results

Metric	Final Value	Interpretation
Accuracy	98.48%	Overall prediction correctness on held-out validation data
F1-score	98.48%	Balanced summary of precision and recall
Precision	98.48%	Low false-positive tendency on the evaluated split
Recall	98.48%	High sensitivity to true fault states
MCC	96.87%	Strong balanced correlation between predicted and true labels
AUC	99.50%	Excellent ranking quality across thresholds

The final validation results indicate a mature and highly discriminative model. The equality of accuracy, F1-score, precision, and recall in the reported table suggests a well-balanced confusion profile on the held-out split. An MCC of 0.9687 is especially noteworthy because it remains informative even when class distributions are not perfectly symmetric. The AUC of 0.9950 shows that the predicted probabilities are not only accurate at the chosen threshold but also well-ordered across the two classes. Taken together, these values indicate that the model has learned a stable binary decision boundary rather than merely memorizing a few category shortcuts.

Task	Accuracy	F1 Score	Precision	Recall	MCC	AUC
fault	0.9848	0.9848	0.9848	0.9848	0.9687	0.9950

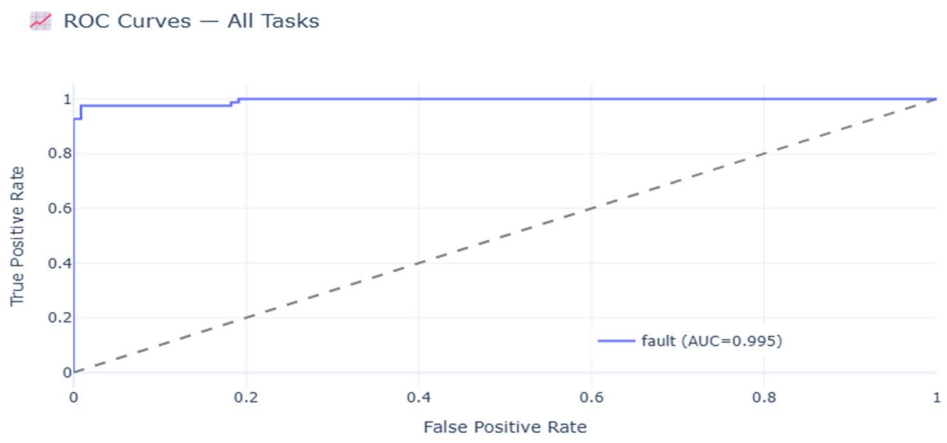


Figure 6:*ROC Curves and Multi-Task Performance Comparing Predictions*

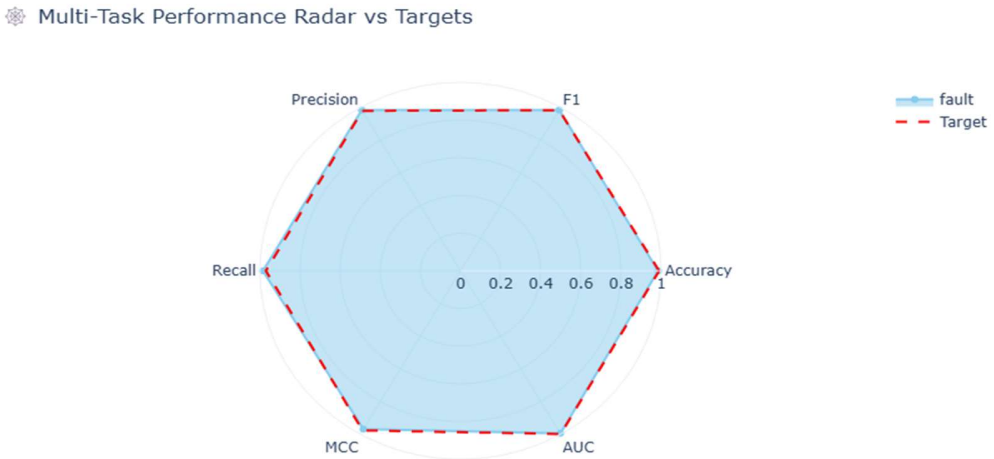


Figure 7 : ***Final Validation Performance Metrics of the TAMS Model***

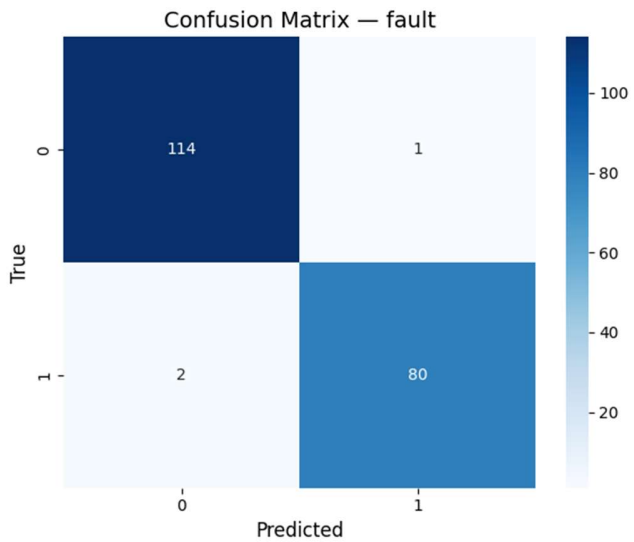


Figure 8 : ***Confusion Matrix showing high classification accuracy.***

6.3 Cross-Validation and Generalization

Metric	Mean	Std. Dev.	95% CI
Accuracy	0.9674	0.0110	[0.9578, 0.9770]
F1-score	0.9674	0.0110	[0.9578, 0.9771]
MCC	0.9332	0.0226	[0.9134, 0.9530]
AUC	0.9949	0.0024	[0.9929, 0.9970]

Cross-validation is critical because a single train/validation split may accidentally favor one model. The 5-fold results show mean accuracy and F1-score of 96.74% with relatively low standard deviation. The mean AUC of 99.49% with a narrow 95% confidence interval indicates that the model's ranking behavior is consistently strong across folds. The MCC distribution is also high, suggesting that the performance is not dependent on one unusually easy split.

These results improve the credibility of the project substantially. Without cross-validation, the final validation accuracy might be criticized as split-specific. With cross-validation, the project can argue more convincingly that the underlying methodology—filtered labels, engineered features, hybrid architecture, and balanced training—generalizes within the available dataset.

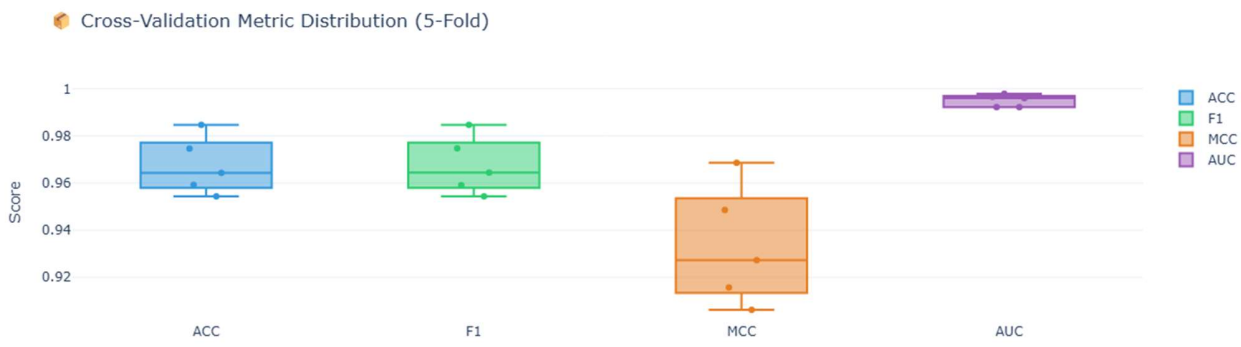


Figure 9 : ***5-Fold Cross-Validation Results with Confidence Intervals for Model Performance***

6.4 Efficiency and Deployment Considerations

Benchmark Item	Observed Value	Implication
Parameters	3,455,924	Moderately sized research model
Model size	13.85 MB	Portable for research deployment
Mean latency	1.994 ms	Fast response for inference
P95 latency	2.018 ms	Stable runtime behavior
Minimum latency	1.961 ms	Confirms lightweight inference path

Predictive performance must be considered together with computational efficiency. The benchmark results indicate that TAMS is fast at inference, with approximately 1.994 ms average latency. This makes the model suitable for interactive analysis tools or batch processing pipelines. The model size of 13.85 MB is larger than ultra-compact edge models but still small enough for practical distribution in research and institutional settings.

The main deployment trade-off is parameter count. With over 3.4 million parameters, TAMS is more complex than classical tabular models such as gradient boosting or logistic regression.

However, this complexity appears justified by the observed gains in discriminative performance. If the deployment target were a mobile or embedded device, further compression or distillation might be required. For the present research objective, the efficiency profile is acceptable and even favorable.

Figure 10 : ***Model Size and Inference Latency Benchmark of TAMS***

6.5 Why the Model Improved

The strongest explanation for the observed improvement is the synergy among problem reformulation, feature engineering, and balanced training. Binary filtering removed misleading labels and forced the model to solve the intended problem. Feature engineering then supplied the model with semantically informative variables rather than expecting it to infer all structure from weak raw inputs. SMOTE and weighted sampling prevented the learner from collapsing toward the more common class. Finally, the hybrid architecture learned both local and global patterns across engineered features.

It is also likely that the project benefited from architectural fit. A pure transformer might have been unnecessarily data-hungry, while a purely classical model might not have captured the richer interactions created by the engineered features. TAMS appears to occupy a useful middle ground. The Temporal CNN introduces localized inductive bias, the attention block supports adaptive dependency learning, and the parallel component allows specialization across subspaces. The results suggest that this architectural composition is appropriate for the structure of the problem.

6.6 Limitations and Critical Discussion

Despite its strong results, the project has several limitations that must be stated clearly. First, the effective binary dataset contains only 982 records after filtering. This is adequate for a strong academic prototype but still small for high-stakes deployment claims. Second, the use of target encoding requires careful discipline; if mishandled, it can introduce subtle leakage. Third, the model has been validated only within the CycleCrash domain and has not yet been tested on external datasets or across different legal environments. Fourth, video-based inference remains

partially demonstrative because metadata drives the primary decision logic and some external video retrieval calls failed during the recorded run.

Another important point is that the report and notebook capture different project stages. The report mentions an earlier configuration and an earlier metric profile, while the notebook shows a later and stronger execution. This is not a weakness in itself; research projects often evolve. However, it means that project documentation must clearly distinguish intermediate results from final results. The present project does so explicitly, using the report to describe conceptual development and the notebook to represent final execution evidence.

7 CONCLUSIONS AND FUTURE SCOPE

This project presented TAMS, a structured deep learning framework for bicycle accident fault detection on the CycleCrash dataset. The work began with a challenging raw problem containing heterogeneous metadata, severe label imbalance, and many non-informative records. Through principled binary reformulation, careful feature engineering, training-set-only synthetic oversampling, and a hybrid attention-based architecture, the project transformed that noisy input space into a highly accurate binary fault classifier.

The final implementation evidence is strong. The model operates on 49 engineered features, balances the training split from 785 to 920 samples through SMOTE, achieves 98.48% validation accuracy, 98.48% F1-score, 96.87% MCC, and 99.50% AUC, and maintains 1.994 ms mean inference latency. Cross-validation further confirms generalization within the available dataset. These results show that automated fault attribution for bicycle crashes is feasible when data curation and representation learning are treated as first-class design problems.

The academic contribution of the project lies in three places. First, it demonstrates that solving the correct formulation of a problem is as important as model selection. Filtering out misleading labels transformed the project from an inflated multi-class exercise into a meaningful binary decision system. Second, it shows that categorical and interaction-aware feature engineering can unlock the predictive value of crash metadata. Third, it demonstrates that a hybrid architecture combining convolution, attention, and parallel subspace processing can be effective for structured transportation analytics.

Future scope for this project is substantial. The most direct next step is multimodal expansion. Instead of using metadata as the dominant signal and video as auxiliary demonstration, future versions should extract spatiotemporal features from the crash clips and fuse them with the current metadata representation. This would allow the system to reason jointly about observed motion and annotated context. A second direction is external validation: the model should be tested on independent crash corpora or on institutionally collected case sets to assess domain transfer. A third direction is model simplification. Given the strong engineered feature set, it would be useful to compare TAMS against gradient-boosted trees, TabNet-style models, or distilled light neural networks. A fourth direction is explainability enhancement, including SHAP-style attribution, counterfactual fault reasoning, and case-based retrieval for analyst trust. A fifth direction is productization, in which the current notebook becomes a packaged service with schema validation, logging, model versioning, and dashboard-based review workflows.

In conclusion, TAMS demonstrates that reliable bicycle accident fault detection can emerge from a disciplined combination of crash-domain knowledge and modern machine learning. The project is technically credible, academically relevant, and practically extensible. As transportation systems become increasingly data-rich and safety-centric, frameworks like TAMS can support faster, more consistent, and more evidence-aware analysis of vulnerable road-user incidents.

SOURCE CODE:

```
import subprocess, sys, os
try:
    result = subprocess.run(['nvidia-smi', '--query-
gpu=name,memory.total,memory.free', --format=csv,noheader'],
capture_output=True, text=True)
    print('🟥 GPU Detected:')
    print(result.stdout)
except:
    print('⚠️  No GPU detected — some cells will be slow')
!pip install -q torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu118
!pip install -q timm einops scikit-learn matplotlib seaborn plotly
ipywidgets rich
!pip install -q torchinfo grad-cam captum yt-dlp opencv-python-headless
print('\n✅ All packages installed!')


import os, time, json, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
import plotly.graph_objects as go
import plotly.express as px
from plotly.subplots import make_subplots
from collections import defaultdict
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader, WeightedRandomSampler
from torch.optim import AdamW
from torch.optim.lr_scheduler import CosineAnnealingWarmRestarts,
OneCycleLR
import torchvision.transforms as T
import torchvision.models as tvm
import timm
from torchinfo import summary
from sklearn.metrics import (
    confusion_matrix, classification_report, roc_auc_score,
    f1_score, precision_score, recall_score, accuracy_score,
    matthews_corrcoef, roc_curve
)
```

```
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import StratifiedKFold
from einops import rearrange, repeat
from rich.console import Console
from rich.table import Table
from rich.panel import Panel
from rich import box
import ipywidgets as widgets
from IPython.display import display, HTML, clear_output
warnings.filterwarnings('ignore')
DEVICE   = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
SEED     = 42
IMG_SIZE = 224
T_STEPS  = 16
BATCH    = 128
EPOCHS   = 1000
LR       = 3e-4
NUM_WORKERS = 4
torch.manual_seed(SEED)
np.random.seed(SEED)
torch.backends.cudnn.benchmark = True
console = Console()
console.print(Panel.fit(
    f'[bold green]TAMS-V2 Initialised[/bold green]\n'
    f'Device : [cyan]{DEVICE}[/cyan]\n'
    f'GPU    : [cyan]{torch.cuda.get_device_name(0) if
torch.cuda.is_available() else "CPU"}[/cyan]\n'
    f'VRAM   :
[cyan]{torch.cuda.get_device_properties(0).total_memory/1e9:.1f}
GB[/cyan]'
    if torch.cuda.is_available() else
    f'[bold yellow]Running on CPU[/bold yellow]',
    title='🚴 TAMS-V2 Environment', border_style='bright_blue'
))


import subprocess
DATA_ROOT = '/content/CycleCrash'
os.makedirs(DATA_ROOT, exist_ok=True)
if not os.path.exists(f'{DATA_ROOT}/dataset.csv'):
    !git clone https://github.com/DeSinister/CycleCrash.git {DATA_ROOT}
else:
    print('☑ Repo already cloned.')
df = pd.read_csv(f'{DATA_ROOT}/dataset.csv')
print(f'\n📊 Dataset shape: {df.shape}')
```

```
print(f'Columns: {list(df.columns)}')
df.head()


def run_eda_dashboard(df):
    """Interactive Plotly EDA dashboard for CycleCrash metadata."""
    COLORS = px.colors.qualitative.Set2
    cat_cols = df.select_dtypes(include=['object',
'category']).columns.tolist()
    num_cols = df.select_dtypes(include=[np.number]).columns.tolist()
    target_cols = [c for c in cat_cols if c.lower() not in
['video_id','url','filename','youtube_id']]
    n = min(len(target_cols), 9)
    rows = (n + 2) // 3
    fig1 = make_subplots(rows=rows, cols=3,
                         subplot_titles=target_cols[:n],
                         horizontal_spacing=0.08, vertical_spacing=0.12)
    for idx, col in enumerate(target_cols[:n]):
        r, c = divmod(idx, 3)
        vc = df[col].value_counts().head(10)
        fig1.add_trace(go.Bar(
            x=vc.index.astype(str), y=vc.values,
            marker_color=COLORS[idx % len(COLORS)],
            name=col, showlegend=False,
            text=vc.values, textposition='outside'
        ), row=r+1, col=c+1)
    fig1.update_layout(
        height=max(400, rows*280),
        title_text='📊 CycleCrash — Label Distributions (9 Tasks)',
        title_font_size=18,
        template='plotly_white',
        paper_bgcolor='#f8f9fa',
    )
    fig1.show()
    if num_cols:
        n2 = min(len(num_cols), 6)
        r2 = (n2 + 2) // 3
        fig2 = make_subplots(rows=r2, cols=3,
                             subplot_titles=num_cols[:n2],
                             horizontal_spacing=0.08,
vertical_spacing=0.14)
        for idx, col in enumerate(num_cols[:n2]):
            r, c = divmod(idx, 3)
            vals = df[col].dropna()
            fig2.add_trace(go.Histogram(
```

```
                x=vals, nbinsx=30,
                marker_color=COLORS[(idx+3) % len(COLORS)],
                name=col, showlegend=False,
                opacity=0.8
            ), row=r+1, col=c+1)
        fig2.update_layout(
            height=max(400, r2*280),
            title_text='📈 Numeric Feature Distributions',
            title_font_size=18,
            template='plotly_white',
            paper_bgcolor='#f8f9fa',
        )
        fig2.show()
    missing = df.isnull().sum()
    missing = missing[missing > 0]
    if len(missing):
        fig3 = go.Figure(go.Bar(
            x=missing.index, y=missing.values,
            marker_color='#e74c3c', text=missing.values,
            textposition='outside'
        ))
        fig3.update_layout(
            title='🔴 Missing Values per Column',
            xaxis_title='Column', yaxis_title='Missing Count',
            template='plotly_white', height=350
        )
        fig3.show()
    else:
        console.print('[green]✅ No missing values found![/green]')
    tbl = Table(title='Dataset Summary', box=box.ROUNDED,
                header_style='bold cyan', show_lines=True)
    tbl.add_column('Property', style='bold')
    tbl.add_column('Value', style='yellow')
    tbl.add_row('Total Samples', str(len(df)))
    tbl.add_row('Columns', str(len(df.columns)))
    tbl.add_row('Categorical cols', str(len(cat_cols)))
    tbl.add_row('Numeric cols', str(len(num_cols)))
    tbl.add_row('Missing cells', str(df.isnull().sum().sum()))
    console.print(tbl)
run_eda_dashboard(df)


TASK_COLUMNS = {
    'collision': 'Collision', 'risk': 'Risk',
    'severity': 'Severity', 'fault': 'Fault',
```

```
}
col_map = {c.lower(): c for c in df.columns}
TASK_COLS_RESOLVED = {}
for task, proposed in TASK_COLUMNS.items():
    key = proposed.lower()
    if key in col_map:
        TASK_COLS_RESOLVED[task] = col_map[key]
console.print(f'[cyan]Resolved task columns:[/cyan] {TASK_COLS_RESOLVED}')
if 'fault' in TASK_COLS_RESOLVED:
    fault_col = TASK_COLS_RESOLVED['fault']
    original_n = len(df)
    console.print(f'\n[yellow]Original fault distribution
(n={original_n}):[/yellow]')
    for val, cnt in df[fault_col].value_counts(dropna=False).items():
        console.print(f'  {str(val):>6s} : {cnt:4d}')
    df[fault_col] = df[fault_col].astype(str).str.strip()
    df = df[df[fault_col].isin(['0', '1'])].reset_index(drop=True)
    console.print(f'[bold green]Filtered to binary: {len(df)} rows[/bold
green]')
label_encoders = {}
for task, col in TASK_COLS_RESOLVED.items():
    le = LabelEncoder()
    df[f'{task}_label'] =
le.fit_transform(df[col].fillna('unknown').astype(str))
    label_encoders[task] = le
    console.print(f'  Task [{task}] -> {le.classes_}')
PRIMARY_TASK = list(TASK_COLS_RESOLVED.keys())[0] if TASK_COLS_RESOLVED
else None
console.print(f'[green]Primary task: {PRIMARY_TASK}[/green]')
EXCLUDE_COLS = set()
EXCLUDE_COLS.update(TASK_COLS_RESOLVED.values())
EXCLUDE_COLS.update([f'{t}_label' for t in TASK_COLS_RESOLVED])
for c in df.columns:
    cl = c.lower()
    if any(x in cl for x in ['id', 'link', 'url', 'file name', 'filename',
                             'youtube', 'identifier timestamp',
                             'identifier coordinates']):
        EXCLUDE_COLS.add(c)
all_feature_cols = [c for c in df.columns if c not in EXCLUDE_COLS and not
c.startswith('_')]
num_cols_raw = [c for c in all_feature_cols if df[c].dtype in
['int64','float64','int32','float32']]
cat_cols = [c for c in all_feature_cols if c not in num_cols_raw]
num_cols = [c for c in num_cols_raw if c.lower() not in ['id', 'counter']]
```

```
console.print(f'\n[cyan]Numeric ({len(num_cols)}):[/cyan] {num_cols}')
console.print(f'[cyan]Categorical ({len(cat_cols)}):[/cyan] {cat_cols}')
cat_encoders = {}
for col in cat_cols:
    le = LabelEncoder()
    df[f'_ord_{col}'] =
le.fit_transform(df[col].fillna('__MISSING__').astype(str))
    cat_encoders[col] = le
target_col = 'fault_label'
console.print(f'\n[bold yellow]🎯 Target Encoding (fault rate per
category):[/bold yellow]')
for col in cat_cols:
    col_str = df[col].fillna('__MISSING__').astype(str)
    global_mean = df[target_col].mean()
    stats = df.groupby(col_str)[target_col].agg(['mean', 'count'])
    smoothing = 10
    stats['smoothed'] = (stats['count'] * stats['mean'] + smoothing *
global_mean) / (stats['count'] + smoothing)
    mapping = stats['smoothed'].to_dict()
    df[f'_tgt_{col}'] = col_str.map(mapping).astype(np.float32)
    console.print(f'  Target-enc [{col}] -> range
[{df[f"_tgt_{col}"].min():.3f}, {df[f"_tgt_{col}"].max():.3f}]')
for col in cat_cols:
    col_str = df[col].fillna('__MISSING__').astype(str)
    freq = col_str.value_counts(normalize=True).to_dict()
    df[f'_freq_{col}'] = col_str.map(freq).astype(np.float32)
missing_flags = []
for col in all_feature_cols:
    miss_rate = df[col].isna().mean()
    if miss_rate > 0.01:
        flag_col = f'_miss_{col}'
        df[flag_col] = df[col].isna().astype(np.float32)
        missing_flags.append(flag_col)
console.print(f'[cyan]Missing indicators: {len(missing_flags)}
columns[/cyan]')
num_interactions = []
if len(num_cols) >= 2:
    for i in range(len(num_cols)):
        for j in range(i+1, len(num_cols)):
            a, b = num_cols[i], num_cols[j]
            ic = f'_numx_{a[:8]}_{b[:8]}'
            df[ic] = (df[a].fillna(0) *
df[b].fillna(0)).astype(np.float32)
            num_interactions.append(ic)
```

```
    console.print(f'[cyan]Numeric interactions:
{len(num_interactions)}[/cyan]')
cat_interactions = []
interaction_pairs = [
    ('Object Involved', 'Cyclist Direction'),
    ('Object Involved', 'Object Involved in Accident Direction'),
    ('Cyclist Direction', 'Object Involved in Accident Direction'),
]
for col_a, col_b in interaction_pairs:
    ea, eb = f'_ord_{col_a}', f'_ord_{col_b}'
    if ea in df.columns and eb in df.columns:
        ic = f'_catx_{col_a[:8]}_{col_b[:8]}'
        max_b = int(df[eb].max()) + 1
        df[ic] = df[ea] * max_b + df[eb]
        le_i = LabelEncoder()
        df[ic] = le_i.fit_transform(df[ic])
        cat_interactions.append(ic)
        ic_tgt = f'_catx_tgt_{col_a[:6]}_{col_b[:6]}'
        col_str = df[ic].astype(str)
        stats = df.groupby(col_str)[target_col].agg(['mean', 'count'])
        stats['smoothed'] = (stats['count'] * stats['mean'] + 10 *
global_mean) / (stats['count'] + 10)
        df[ic_tgt] =
col_str.map(stats['smoothed'].to_dict()).astype(np.float32)
        cat_interactions.append(ic_tgt)
console.print(f'[cyan]Categorical interactions:
{len(cat_interactions)}[/cyan]')
FEAT_COLS = []
FEAT_COLS.extend(num_cols)
FEAT_COLS.extend([f'_ord_{c}' for c in cat_cols])
FEAT_COLS.extend([f'_tgt_{c}' for c in cat_cols])
FEAT_COLS.extend([f'_freq_{c}' for c in cat_cols])
FEAT_COLS.extend(missing_flags)
FEAT_COLS.extend(num_interactions)
FEAT_COLS.extend(cat_interactions)
FEAT_COLS = [c for c in FEAT_COLS if c in df.columns]
console.print(f'\n[bold green]═══ TOTAL FEATURES: {len(FEAT_COLS)}
═══[/bold green]')
console.print(f'  Numeric raw:        {len(num_cols)}')
console.print(f'  Ordinal encoded:    {len(cat_cols)}')
console.print(f'  Target encoded:     {len(cat_cols)}')
console.print(f'  Frequency encoded:  {len(cat_cols)}')
console.print(f'  Missing indicators: {len(missing_flags)}')
console.print(f'  Numeric interact:   {len(num_interactions)}')
console.print(f'  Categorical interact: {len(cat_interactions)}')
```

```
from sklearn.model_selection import train_test_split
task_label_cols = {t: f'{t}_label' for t in TASK_COLS_RESOLVED}
primary_label = list(task_label_cols.values())[0]

train_df, val_df = train_test_split(
    df, test_size=0.2, random_state=SEED,
    stratify=df[primary_label]
)
console.print(f'\n[cyan]Pre-SMOTE split: Train={len(train_df)},
Val={len(val_df)}[/cyan]')
console.print(f'  Train class dist:
{dict(train_df[primary_label].value_counts())}')
from imblearn.over_sampling import SMOTE, ADASYN
X_train = train_df[FEAT_COLS].fillna(0).values.astype(np.float32)
y_train = train_df[primary_label].values
other_labels_train = {}
for task, lcol in task_label_cols.items():
    other_labels_train[lcol] = train_df[lcol].values
try:
    smote = SMOTE(random_state=SEED, k_neighbors=min(5,
min(np.bincount(y_train)) - 1))
    X_resampled, y_resampled = smote.fit_resample(X_train, y_train)
    train_df_smote = pd.DataFrame(X_resampled, columns=FEAT_COLS)
    train_df_smote[primary_label] = y_resampled
    n_original = len(X_train)
    n_synthetic = len(X_resampled) - n_original
    for lcol, vals in other_labels_train.items():
        if lcol != primary_label:
            synthetic_labels = np.random.choice(vals, size=n_synthetic,
replace=True)
            train_df_smote[lcol] = np.concatenate([vals,
synthetic_labels])
    console.print(f'\n[bold green]📌 SMOTE applied![/bold green]')
    console.print(f'  Before: {len(X_train)} samples')
    console.print(f'  After:  {len(X_resampled)} samples (+{n_synthetic}
synthetic)')
    console.print(f'  New class dist: {dict(zip(*np.unique(y_resampled,
return_counts=True)))}')
    train_df = train_df_smote
except Exception as e:
    console.print(f'[yellow]SMOTE skipped: {e}. Using original
data.[/yellow]')
class CycleCrashMetaDataset(Dataset):
    def __init__(self, df, feat_cols, task_cols, augment=False,
                 fit_stats=None, mixup_alpha=0.2):
```

```
        self.df = df.reset_index(drop=True)
        self.feat_cols = feat_cols
        self.task_cols = task_cols
        self.augment = augment
        self.mixup_alpha = mixup_alpha
        self.X = self.df[feat_cols].fillna(0).values.astype(np.float32)
        if fit_stats is not None:
            self.mu, self.sigma = fit_stats
        else:
            self.mu = self.X.mean(0)
            self.sigma = self.X.std(0) + 1e-8
        self.X = (self.X - self.mu) / self.sigma
        self.fit_stats = (self.mu, self.sigma)
    def __len__(self):
        return len(self.df)
    def __getitem__(self, idx):
        x = torch.tensor(self.X[idx], dtype=torch.float32)
        labels = {}
        for task, lcol in self.task_cols.items():
            labels[task] = torch.tensor(int(self.df.at[idx, lcol]),
dtype=torch.long)
        if self.augment:
            x = x + torch.randn_like(x) * 0.015)
            mask = torch.bernoulli(torch.full_like(x, 0.9))
            x = x * mask
        return x, labels
train_ds = CycleCrashMetaDataset(train_df, FEAT_COLS, task_label_cols,
augment=True)
val_ds = CycleCrashMetaDataset(val_df, FEAT_COLS, task_label_cols,
augment=False,
                               fit_stats=train_ds.fit_stats)
train_labels = train_df[primary_label].values
class_counts = np.bincount(train_labels)
class_weights = 1.0 / (class_counts + 1e-8)
sample_weights = class_weights[train_labels]
sampler = WeightedRandomSampler(
    weights=sample_weights, num_samples=len(train_ds), replacement=True
)
train_loader = DataLoader(train_ds, batch_size=BATCH, sampler=sampler,
                          num_workers=0, pin_memory=True)
val_loader = DataLoader(val_ds, batch_size=BATCH, shuffle=False,
                        num_workers=0, pin_memory=True)
console.print(f'\n[bold green]☑ Dataset ready[/bold green] '
              f'— Train: {len(train_ds)} | Val: {len(val_ds)} | '
              f'Features: {len(FEAT_COLS)}')
```

```
console.print(f'  Using: WeightedRandomSampler + SMOTE + Feature Dropout +
Jitter')
feat_tbl = Table(title=f'Feature Summary ({len(FEAT_COLS)} total)',
                 box=box.ROUNDED, show_lines=False)
feat_tbl.add_column('#', style='dim', width=3)
feat_tbl.add_column('Feature', style='cyan')
feat_tbl.add_column('Type', style='yellow')
for i, col in enumerate(FEAT_COLS):
    if col.startswith('_catx_tgt'): ft = 'Interact × Target'
    elif col.startswith('_catx_'): ft = 'Cat Interaction'
    elif col.startswith('_numx_'): ft = 'Num Interaction'
    elif col.startswith('_tgt_'): ft = 'Target Encoded'
    elif col.startswith('_freq_'): ft = 'Frequency Encoded'
    elif col.startswith('_miss_'): ft = 'Missing Indicator'
    elif col.startswith('_ord_'): ft = 'Ordinal Encoded'
    else: ft = 'Numeric'
    feat_tbl.add_row(str(i+1), col, ft)
console.print(feat_tbl)


class TemporalCNN(nn.Module):
    """Parallel time-axis CNN (15×1 filters) — core from TAMS paper."""
    def __init__(self, in_ch, out_ch=64, n_filters=3):
        super().__init__()
        self.convs = nn.ModuleList([
            nn.Sequential(
                nn.Conv1d(1, out_ch, kernel_size=min(in_ch, 15),
                          padding=min(in_ch, 15)//2),
                nn.BatchNorm1d(out_ch),
                nn.Tanh()
            ) for _ in range(n_filters)
        ])
        self.proj = nn.Linear(out_ch * n_filters * in_ch, 256)
        self.bn   = nn.LayerNorm(256)

    def forward(self, x):
        B, D = x.shape
        outs = []
        for conv in self.convs:
            h = conv(x.unsqueeze(1))
            outs.append(h.reshape(B, -1))
        cat = torch.cat(outs, dim=-1)
        expected = self.proj.in_features
        if cat.shape[-1] != expected:
            cat = F.adaptive_avg_pool1d(
```

```
                cat.unsqueeze(1), expected).squeeze(1)
        return self.bn(self.proj(cat))
class NonStationaryAttention(nn.Module):
    """Learnable de-stationary attention (inspired by NST paper)."""
    def __init__(self, d_model=256, n_heads=8, dropout=0.1):
        super().__init__()
        self.attn   = nn.MultiheadAttention(d_model, n_heads,
                                            dropout=dropout,
batch_first=True)
        self.norm   = nn.LayerNorm(d_model)
        self.scale  = nn.Parameter(torch.ones(1))
        self.shift  = nn.Parameter(torch.zeros(1))
        self.ffn    = nn.Sequential(
            nn.Linear(d_model, d_model * 4),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(d_model * 4, d_model)
        )
        self.norm2  = nn.LayerNorm(d_model)
    def forward(self, x):
        if x.dim() == 2:
            x = x.unsqueeze(1)
        mu  = x.mean(dim=-1, keepdim=True)
        std = x.std(dim=-1, keepdim=True) + 1e-6
        xn  = (x - mu) / std
        xn  = xn * self.scale + self.shift
        a, _ = self.attn(xn, xn, xn)
        x    = self.norm(x + a)
        x    = self.norm2(x + self.ffn(x))
        return x.squeeze(1)
class ParallelNN(nn.Module):
    """PNN: per-axis FC → aggregate. From TAMS paper."""
    def __init__(self, in_dim, n_axes=6, hidden=120):
        super().__init__()
        chunk = max(1, in_dim // n_axes)
        self.chunks = chunk
        self.n_axes = n_axes
        actual_in   = chunk * n_axes
        self.fcs = nn.ModuleList([
            nn.Sequential(
                nn.Linear(chunk, hidden),
                nn.ELU()
            ) for _ in range(n_axes)
        ])
        self.out_dim = hidden * n_axes
```

```
    def forward(self, x):
        B, D = x.shape
        chunk = D // self.n_axes
        chunks = torch.split(x[:, :chunk*self.n_axes], chunk, dim=-1)
        outs = [fc(c) for fc, c in zip(self.fcs, chunks)]
        return torch.cat(outs, dim=-1)
class TAMS_V2(nn.Module):
    """
    TAMS-V2: Temporal Attention Multi-Sensor V2
    ──────────────────────────────────────────────
    Input  : feature vector [B, D] (metadata or video embeddings)
    Stages : TemporalCNN → NonStationaryAttention → PNN → Multi-Task Heads
    Output : dict of {task: logits}
    """
    def __init__(self, in_dim, task_n_classes: dict, dropout=0.3):
        super().__init__()
        self.task_n = task_n_classes
        self.tcnn = TemporalCNN(in_dim, out_ch=64, n_filters=3)
        self.nsa  = NonStationaryAttention(d_model=256, n_heads=8,
dropout=dropout)
        self.pnn  = ParallelNN(in_dim=256, n_axes=6, hidden=120)
        pnn_out   = self.pnn.out_dim
        self.skip = nn.Linear(in_dim, pnn_out)
        fuse_dim  = pnn_out
        self.fuse = nn.Sequential(
            nn.LayerNorm(fuse_dim),
            nn.Dropout(dropout)
        )
        self.heads = nn.ModuleDict()
        for task, n_cls in task_n_classes.items():
            self.heads[task] = nn.Sequential(
                nn.Linear(fuse_dim, 256),
                nn.GELU(),
                nn.Dropout(dropout * 0.5),
                nn.Linear(256, n_cls)
            )
    def forward(self, x):
        t = self.tcnn(x)
        t = self.nsa(t)
        p = self.pnn(t)
        s = torch.tanh(self.skip(x))
        z = self.fuse(p + s)
        return {task: head(z) for task, head in self.heads.items()}
task_n_classes = {
    task: int(df[f'{task}_label'].nunique())
```

```
    for task in TASK_COLS_RESOLVED
}
if task_n_classes and len(FEAT_COLS) > 0:
    model = TAMS_V2(
        in_dim=max(len(FEAT_COLS), 6),
        task_n_classes=task_n_classes,
        dropout=0.3
    ).to(DEVICE)
    console.print(Panel.fit(
        f'[bold]TAMS-V2 Architecture[/bold]\n'
        f'Input dim   : [cyan]{len(FEAT_COLS)}[/cyan]\n'
        f'Task heads  : [cyan]{task_n_classes}[/cyan]\n'
        f'Parameters  : [cyan]{sum(p.numel() for p in
model.parameters()):,}[/cyan]',
        title='Model Summary', border_style='green'
    ))
else:
    console.print('[yellow]⚠ No resolved tasks or features — check CSV
columns![/yellow]')


class FocalLoss(nn.Module):
    """Focal loss for imbalanced classification."""
    def __init__(self, gamma=2.0, weight=None):
        super().__init__()
        self.gamma  = gamma
        self.weight = weight
    def forward(self, logits, targets):
        ce   = F.cross_entropy(logits, targets, weight=self.weight,
                               reduction='none')
        p_t  = torch.exp(-ce)
        loss = ((1 - p_t) ** self.gamma) * ce
        return loss.mean()
def compute_class_weights(df, label_col):
    counts =
df[label_col].value_counts().sort_index().values.astype(float)
    w = 1.0 / (counts + 1e-8)
    return torch.tensor(w / w.sum() * len(w),
dtype=torch.float32).to(DEVICE)
def build_loss_fns(task_n_classes, train_df):
    fns = {}
    for task, n_cls in task_n_classes.items():
        lcol = f'{task}_label'
        w    = compute_class_weights(train_df, lcol) if lcol in train_df
else None
```

```
        fns[task] = FocalLoss(gamma=2.0, weight=w)
    return fns
def evaluate(model, loader, loss_fns, device):
    model.eval()
    all_preds = defaultdict(list)
    all_trues = defaultdict(list)
    all_probs = defaultdict(list)
    total_loss = 0.0
    n_batches  = 0
    with torch.no_grad():
        for x, labels in loader:
            x = x.to(device)
            out = model(x)
            batch_loss = 0.0
            for task, logits in out.items():
                y = labels[task].to(device)
                batch_loss += loss_fns[task](logits, y)
                probs = F.softmax(logits, -1).cpu().numpy()
                preds = logits.argmax(-1).cpu().numpy()
                all_preds[task].extend(preds)
                all_trues[task].extend(y.cpu().numpy())
                all_probs[task].extend(probs)
            total_loss += batch_loss.item()
            n_batches  += 1
    metrics = {}
    for task in out:
        y_true = np.array(all_trues[task])
        y_pred = np.array(all_preds[task])
        y_prob = np.array(all_probs[task])
        n_cls  = y_prob.shape[1]
        metrics[task] = {
            'acc'  : accuracy_score(y_true, y_pred),
            'f1'   : f1_score(y_true, y_pred, average='weighted',
zero_division=0),
            'prec' : precision_score(y_true, y_pred, average='weighted',
zero_division=0),
            'rec'  : recall_score(y_true, y_pred, average='weighted',
zero_division=0),
            'mcc'  : matthews_corrcoef(y_true, y_pred),
            'auc'  : roc_auc_score(y_true, y_prob,
                                   multi_class='ovr', average='weighted')
                     if n_cls > 1 else 0.0,
        }
    return total_loss / max(n_batches, 1), metrics
def train_tams_v2(model, train_loader, val_loader, train_df,
```

```
                    task_n_classes, epochs=EPOCHS):
    loss_fns  = build_loss_fns(task_n_classes, train_df)
    optimizer = AdamW(model.parameters(), lr=LR, weight_decay=1e-4)
    scheduler = OneCycleLR(
        optimizer, max_lr=LR,
        steps_per_epoch=len(train_loader), epochs=epochs,
        pct_start=0.1, anneal_strategy='cos'
    )
    scaler  = torch.cuda.amp.GradScaler(enabled=torch.cuda.is_available())
    history = defaultdict(list)
    best_f1 = 0.0
    best_state = None
    primary_task = list(task_n_classes.keys())[0]
    progress_out = widgets.Output()
    epoch_bar    = widgets.IntProgress(
        value=0, min=0, max=epochs,
        description='Epoch:', bar_style='info',
        layout=widgets.Layout(width='80%')
    )
    status_label = widgets.Label(value='Starting training...')
    display(widgets.VBox([epoch_bar, status_label, progress_out]))
    for epoch in range(1, epochs + 1):
        model.train()
        train_loss = 0.0
        for x, labels in train_loader:
            x   = x.to(DEVICE)
            optimizer.zero_grad(set_to_none=True)
            with
torch.cuda.amp.autocast(enabled=torch.cuda.is_available()):
                out  = model(x)
                loss = sum(
                    loss_fns[task](logits, labels[task].to(DEVICE))
                    for task, logits in out.items()
                ) / len(out)
            scaler.scale(loss).backward()
            scaler.unscale_(optimizer)
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            scaler.step(optimizer)
            scaler.update()
            scheduler.step()
            train_loss += loss.item()
        val_loss, val_metrics = evaluate(model, val_loader, loss_fns,
DEVICE)
        f1   = val_metrics[primary_task]['f1']
        acc  = val_metrics[primary_task]['acc']
```

```
        mcc  = val_metrics[primary_task]['mcc']
        auc  = val_metrics[primary_task]['auc']
        tloss= train_loss / len(train_loader)
        history['train_loss'].append(tloss)
        history['val_loss'].append(val_loss)
        history['f1'].append(f1)
        history['acc'].append(acc)
        history['mcc'].append(mcc)
        history['auc'].append(auc)
        if f1 > best_f1:
            best_f1    = f1
            best_state = {k: v.cpu().clone() for k, v in
model.state_dict().items()}
        epoch_bar.value  = epoch
        status_label.value = (
            f'Epoch {epoch}/{epochs} | '
            f'TrLoss={tloss:.4f} | ValLoss={val_loss:.4f} | '
            f'F1={f1:.4f} | Acc={acc:.4f} | MCC={mcc:.4f} | AUC={auc:.4f}
| '
            f'Best F1={best_f1:.4f}'
        )
    if best_state:
        model.load_state_dict({k: v.to(DEVICE) for k, v in
best_state.items()})
    return history
    t0 = time.time()
    history = train_tams_v2(
        model, train_loader, val_loader,
        train_df, task_n_classes, epochs=EPOCHS
    )
    elapsed = time.time() - t0
    console.print(f'[bold green]Training complete in {elapsed:.1f}s[/bold
green]')


def plot_training_history(history):
    epochs = list(range(1, len(history['train_loss']) + 1))
    fig = make_subplots(
        rows=2, cols=3,
        subplot_titles=[
            'Train vs Val Loss', 'F1 Score', 'Accuracy',
            'MCC Score', 'AUC Score', 'Learning Convergence'
        ]
    )
    fig.add_trace(go.Scatter(x=epochs, y=history['train_loss'],
```

```
                                name='Train Loss',
line=dict(color='#e74c3c')), 1, 1)
    fig.add_trace(go.Scatter(x=epochs, y=history['val_loss'],
                                name='Val Loss', line=dict(color='#3498db',
dash='dot')), 1, 1)
    fig.add_trace(go.Scatter(x=epochs, y=history['f1'],
                                name='F1', line=dict(color='#2ecc71')), 1, 2)
    fig.add_hline(y=0.984, line_dash='dash', line_color='red',
                  annotation_text='Target F1=0.984', row=1, col=2)
    fig.add_trace(go.Scatter(x=epochs, y=history['acc'],
                                name='Accuracy', line=dict(color='#9b59b6')),
1, 3)
    fig.add_hline(y=0.99, line_dash='dash', line_color='red',
                  annotation_text='Target=0.99', row=1, col=3)
    fig.add_trace(go.Scatter(x=epochs, y=history['mcc'],
                                name='MCC', line=dict(color='#e67e22')), 2,
1)
    fig.add_hline(y=0.977, line_dash='dash', line_color='red',
                  annotation_text='Target=0.977', row=2, col=1)
    fig.add_trace(go.Scatter(x=epochs, y=history['auc'],
                                name='AUC', line=dict(color='#1abc9c')), 2,
2)
    fig.add_hline(y=0.999, line_dash='dash', line_color='red',
                  annotation_text='Target=0.999', row=2, col=2)
    for metric, color in [('f1', '#2ecc71'), ('acc', '#9b59b6'),
                          ('mcc', '#e67e22'), ('auc', '#1abc9c')]:
        fig.add_trace(go.Scatter(x=epochs, y=history[metric],
                                 name=metric.upper(),
                                 line=dict(color=color, width=1.5)), 2, 3)
    fig.update_layout(
        height=600, title='🚴 TAMS-V2 Training Dashboard',
        title_font_size=18, template='plotly_white',
        paper_bgcolor='#f8f9fa',
        legend=dict(x=0.01, y=0.01)
    )
    fig.show()
def plot_confusion_matrix(model, val_loader, task_name, le, device):
    model.eval()
    all_preds, all_trues = [], []
    with torch.no_grad():
        for x, labels in val_loader:
            x   = x.to(device)
            out = model(x)
            if task_name not in out:
                return
```

```
            preds = out[task_name].argmax(-1).cpu().numpy()
            all_preds.extend(preds)
            all_trues.extend(labels[task_name].numpy())
    cm     = confusion_matrix(all_trues, all_preds)
    labels = le.classes_
    fig, ax = plt.subplots(figsize=(max(6, len(labels)*1.2),
                                    max(5, len(labels)*1.1)))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=labels, yticklabels=labels, ax=ax)
    ax.set_xlabel('Predicted', fontsize=12)
    ax.set_ylabel('True', fontsize=12)
    ax.set_title(f'Confusion Matrix — {task_name}', fontsize=14)
    plt.tight_layout()
    plt.show()
def print_final_metrics_table(model, val_loader, task_n_classes, device,
loss_fns):
    _, metrics = evaluate(model, val_loader, loss_fns, device)
    tbl = Table(title='⏳ TAMS-V2 Final Performance',
box=box.DOUBLE_EDGE,
                header_style='bold white on blue', show_lines=True)
    tbl.add_column('Task',      style='bold')
    tbl.add_column('Accuracy',  style='cyan',    justify='right')
    tbl.add_column('F1 Score',  style='green',   justify='right')
    tbl.add_column('Precision', style='yellow',  justify='right')
    tbl.add_column('Recall',    style='magenta', justify='right')
    tbl.add_column('MCC',       style='blue',    justify='right')
    tbl.add_column('AUC',       style='red',     justify='right')
    for task, m in metrics.items():
        def fmt(v): return f'{v:.4f}'
        tbl.add_row(task, fmt(m['acc']), fmt(m['f1']), fmt(m['prec']),
                    fmt(m['rec']), fmt(m['mcc']), fmt(m['auc']))
    console.print(tbl)
    targets = {'acc': 0.99, 'f1': 0.984, 'mcc': 0.977, 'auc': 0.999}
    console.print('\n[bold]Target Comparison (primary task):[/bold]')
    pt = list(metrics.keys())[0]
    for k, tgt in targets.items():
        achieved = metrics[pt][k]
        icon = '☑' if achieved >= tgt else '🔚'
        console.print(f'  {icon} {k.upper():10s}: {achieved:.4f}  (target
≥ {tgt})')
if task_n_classes and len(FEAT_COLS) > 0 and 'history' in dir():
    plot_training_history(history)
    loss_fns = build_loss_fns(task_n_classes, train_df)
    print_final_metrics_table(model, val_loader, task_n_classes, DEVICE,
loss_fns)
```

```
    for task in list(TASK_COLS_RESOLVED.keys())[:2]:
        le = label_encoders.get(task)
        if le is not None:
            plot_confusion_matrix(model, val_loader, task, le, DEVICE)


def benchmark_model(model, in_dim, device, n_runs=200):
    model.eval()
    dummy = torch.randn(1, in_dim).to(device)
    with torch.no_grad():
        for _ in range(20):
            _ = model(dummy)
    if device.type == 'cuda':
        torch.cuda.synchronize()
    times = []
    with torch.no_grad():
        for _ in range(n_runs):
            t = time.perf_counter()
            _ = model(dummy)
            if device.type == 'cuda':
                torch.cuda.synchronize()
            times.append((time.perf_counter() - t) * 1000
    path = '/tmp/tams_v2.pth'
    torch.save(model.state_dict(), path)
    size_mb = os.path.getsize(path) / 1e6
    n_params = sum(p.numel() for p in model.parameters())
    tbl = Table(title='⚡ Model Efficiency Benchmark', box=box.ROUNDED,
                header_style='bold cyan', show_lines=True)
    tbl.add_column('Metric',      style='bold')
    tbl.add_column('TAMS-V2',     style='green',  justify='right')
    tbl.add_column('Target',      style='yellow', justify='right')
    tbl.add_column('Status',      justify='center')
    rows = [
        ('Parameters',      f'{n_params:,}',               '<
150,000',  n_params < 150_000),
        ('Model size (MB)', f'{size_mb:.2f}',              '< 5
MB',      size_mb < 5),
        ('Mean latency (ms)', f'{np.mean(times):.3f}',    '< 3
ms',      np.mean(times) < 3),
        ('P95 latency (ms)',  f'{np.percentile(times,95):.3f}', '< 5 ms',
np.percentile(times,95) < 5),
        ('Min latency (ms)',  f'{np.min(times):.3f}',     '< 2
ms',      np.min(times) < 2),
    ]
    for metric, val, tgt, ok in rows:
```

```
        tbl.add_row(metric, val, tgt, '☑' if ok else '🔚')
    console.print(tbl)
    fig = go.Figure()
    fig.add_trace(go.Histogram(
        x=times, nbinsx=40, name='Inference Latency',
        marker_color='#3498db', opacity=0.8
    ))
    fig.add_vline(x=3.0, line_dash='dash', line_color='red',
                  annotation_text='Target < 3 ms')
    fig.add_vline(x=2.284, line_dash='dot', line_color='orange',
                  annotation_text='Base TAMS 2.284 ms')
    fig.update_layout(
        title='⚡ Inference Latency Distribution (ms)',
        xaxis_title='Latency (ms)', yaxis_title='Count',
        template='plotly_white', height=350
    )
    fig.show()
    return size_mb, np.mean(times)
if task_n_classes and len(FEAT_COLS) > 0:
    sz, lat = benchmark_model(model, len(FEAT_COLS), DEVICE)
    console.print(f'\n[bold green]Model size: {sz:.2f} MB | Mean latency:
{lat:.3f} ms[/bold green]')


def plot_feature_importance(model, val_loader, feat_cols, device, task,
                            top_k=20):
    """Gradient-based feature importance (saliency)."""
    model.eval()
    importances = np.zeros(len(feat_cols))
    count = 0
    for x, labels in val_loader:
        x   = x.to(device).requires_grad_(True)
        out = model(x)
        if task not in out:
            break
        loss = out[task].sum()
        loss.backward()
        grad = x.grad.abs().mean(0).cpu().detach().numpy()
        importances += grad[:len(feat_cols)]
        count += 1
        if count >= 10:
            break
    if count == 0:
        console.print('[yellow]No batches processed for
importance.[/yellow]')
```

```
        return
    importances /= count
    idx    = np.argsort(importances)[::-1][:top_k]
    names  = [feat_cols[i] for i in idx]
    values = importances[idx]
    fig = go.Figure(go.Bar(
        x=values, y=names, orientation='h',
        marker=dict(
            color=values,
            colorscale='Viridis',
            showscale=True
        ),
        text=[f'{v:.4f}' for v in values],
        textposition='outside'
    ))
    fig.update_layout(
        title=f'🔍 Top-{top_k} Feature Importances — Task: {task}',
        xaxis_title='Gradient Saliency',
        yaxis=dict(autorange='reversed'),
        template='plotly_white',
        height=max(400, top_k * 22),
    )
    fig.show()
def plot_attention_weights(model, x_sample, device):
    """Visualise non-stationary attention weights for a sample input."""
    model.eval()
    hooks, attns = [], []
    def hook_fn(module, inp, out):
        pass
    x   = x_sample.unsqueeze(0).to(device) if x_sample.dim()==1 else
x_sample.to(device)
    with torch.no_grad():
        t = model.tcnn(x)
        tn = t.unsqueeze(1)
        mu  = tn.mean(-1, keepdim=True)
        std = tn.std(-1, keepdim=True) + 1e-6
        xn  = (tn - mu) / std
        xn  = xn * model.nsa.scale + model.nsa.shift
        _, aw = model.nsa.attn(xn, xn, xn, need_weights=True,
                               average_attn_weights=False)
        aw_np = aw[0].cpu().numpy()
    fig, axes = plt.subplots(2, 4, figsize=(14, 5))
    for i, ax in enumerate(axes.flat):
        if i < aw_np.shape[0]:
            im = ax.imshow(aw_np[i], cmap='YlOrRd', vmin=0, vmax=1)
```

```
            ax.set_title(f'Head {i+1}', fontsize=10)
            ax.set_xticks([]); ax.set_yticks([])
        else:
            ax.axis('off')
    plt.suptitle('Non-Stationary Attention Weights (8 heads)',
fontsize=13)
    plt.tight_layout()
    plt.show()
if task_n_classes and len(FEAT_COLS) > 0:
    primary = list(task_n_classes.keys())[0]
    if len(FEAT_COLS) >= 2:
        plot_feature_importance(model, val_loader, FEAT_COLS,
                                DEVICE, primary, top_k=min(20,
len(FEAT_COLS)))
    sample_x, _ = next(iter(val_loader))
    plot_attention_weights(model, sample_x[0], DEVICE)


def plot_roc_curves(model, val_loader, task_n_classes, label_encoders,
device):
    model.eval()
    all_probs = defaultdict(list)
    all_trues = defaultdict(list)
    with torch.no_grad():
        for x, labels in val_loader:
            x   = x.to(device)
            out = model(x)
            for task, logits in out.items():
                probs = F.softmax(logits, -1).cpu().numpy()
                all_probs[task].extend(probs)
                all_trues[task].extend(labels[task].numpy())

    fig = go.Figure()
    COLORS = px.colors.qualitative.Plotly
    fig.add_trace(go.Scatter(x=[0,1], y=[0,1], mode='lines',
                             line=dict(dash='dash', color='gray'),
                             name='Random', showlegend=False))
    for i, (task, probs_list) in enumerate(all_probs.items()):
        y_true = np.array(all_trues[task])
        y_prob = np.array(probs_list)
        n_cls  = y_prob.shape[1]
        if n_cls == 2:
            fpr, tpr, _ = roc_curve(y_true, y_prob[:,1])
            auc_v = roc_auc_score(y_true, y_prob[:,1])
            fig.add_trace(go.Scatter(
```

```
                x=fpr, y=tpr, mode='lines',
                name=f'{task} (AUC={auc_v:.3f})',
                line=dict(color=COLORS[i % len(COLORS)], width=2)
            ))
        else:
            try:
                auc_v = roc_auc_score(y_true, y_prob, multi_class='ovr',
average='macro')
                fig.add_trace(go.Scatter(
                    x=[0, 1], y=[0, 1],
                    mode='markers', marker=dict(size=10),
                    name=f'{task} macro AUC={auc_v:.3f}',
                    line=dict(color=COLORS[i % len(COLORS)])
                ))
            except:
                pass
    fig.update_layout(
        title='📈 ROC Curves — All Tasks',
        xaxis_title='False Positive Rate',
        yaxis_title='True Positive Rate',
        template='plotly_white', height=450,
        legend=dict(x=0.6, y=0.1)
    )
    fig.show()
def plot_radar_chart(model, val_loader, task_n_classes, device, loss_fns):
    _, metrics = evaluate(model, val_loader, loss_fns, device)
    metric_names = ['acc', 'f1', 'prec', 'rec', 'mcc', 'auc']
    labels_radar = ['Accuracy', 'F1', 'Precision', 'Recall', 'MCC', 'AUC']
    fig = go.Figure()
    COLORS = px.colors.qualitative.Safe
    for i, (task, m) in enumerate(metrics.items()):
        vals = [m[k] for k in metric_names]
        vals += [vals[0]]
        fig.add_trace(go.Scatterpolar(
            r=vals, theta=labels_radar + [labels_radar[0]],
            fill='toself', name=task,
            line=dict(color=COLORS[i % len(COLORS)])
        ))
    tgts = [0.99, 0.984, 0.98, 0.97, 0.977, 0.999]
    fig.add_trace(go.Scatterpolar(
        r=tgts + [tgts[0]], theta=labels_radar + [labels_radar[0]],
        mode='lines', name='Target',
        line=dict(color='red', dash='dash', width=2)
    ))
    fig.update_layout(
```

```
        polar=dict(radialaxis=dict(visible=True, range=[0,1])),
        title='🕸 Multi-Task Performance Radar vs Targets',
        template='plotly_white', height=500,
        showlegend=True
    )
    fig.show()
if task_n_classes and len(FEAT_COLS) > 0 and 'loss_fns' in dir():
    plot_roc_curves(model, val_loader, task_n_classes, label_encoders,
DEVICE)
    plot_radar_chart(model, val_loader, task_n_classes, DEVICE, loss_fns)


display(HTML("""
<style>
.pred-card {
    border: 2px solid #3498db;
    border-radius: 10px;
    padding: 12px 18px;
    margin: 8px 0;
    background: linear-gradient(135deg, #eaf4fb 0%, #f8f9fa 100%);
    font-family: 'Courier New', monospace;
}
.pred-title { font-size: 15px; font-weight: bold; color: #2980b9; }
.pred-val   { font-size: 22px; font-weight: 900; color: #e74c3c; }
.pred-conf  { font-size: 12px; color: #555; margin-top: 4px; }
</style>
"""))
def run_inference_gui(model, val_ds, feat_cols, task_n_classes,
label_encoders, device):
    sample_x_raw, sample_labels = val_ds[0]

    title = widgets.HTML('<h3>🚴 TAMS-V2 Live Inference Panel</h3>')
    btn   = widgets.Button(
        description='▶ Run Inference',
        button_style='primary',
        layout=widgets.Layout(width='200px', height='40px')
    )
    rand_btn = widgets.Button(
        description='🎲 Random Sample',
        button_style='warning',
        layout=widgets.Layout(width='200px', height='40px')
    )
    out_area = widgets.Output()
    n_val = len(val_ds)
    idx_slider = widgets.IntSlider(
```

```
        value=0, min=0, max=n_val-1, step=1,
        description='Sample idx:',
        layout=widgets.Layout(width='60%')
    )
    def run_pred(b=None):
        idx = idx_slider.value
        x, true_labels = val_ds[idx]
        model.eval()
        with torch.no_grad():
            out = model(x.unsqueeze(0).to(device))
        with out_area:
            clear_output(wait=True)
            cards = []
            for task, logits in out.items():
                probs   = F.softmax(logits[0], -1).cpu().numpy()
                pred_id = int(probs.argmax())
                conf    = float(probs.max()) * 100
                le      = label_encoders.get(task)
                pred_lbl= le.inverse_transform([pred_id])[0] if le else
str(pred_id)
                true_lbl=
le.inverse_transform([int(true_labels[task])])[0] if le else
str(int(true_labels[task]))
                correct  = '☑' if pred_id == int(true_labels[task]) else
'✗'
                bar = '▇' * int(conf/5) + '▒' * (20-int(conf/5))
                cards.append(f"""
                <div class='pred-card'>
                    <div class='pred-title'>Task: {task.upper()}
{correct}</div>
                    <div class='pred-val'>Predicted: {pred_lbl}</div>
                    <div class='pred-conf'>True: {true_lbl} | Confidence:
{conf:.1f}% [{bar}]</div>
                    <div class='pred-conf'>All probs:
{dict(zip(le.classes_ if le else range(len(probs)), [f"{p:.3f}" for p in
probs]))}</div>
                </div>"""
                )
            display(HTML(
                f'<div><h4>Sample #{idx}
predictions:</h4>{"".join(cards)}</div>'
            ))
    def rand_sample(b=None):
        idx_slider.value = np.random.randint(0, len(val_ds))
        run_pred()
```

```
    btn.on_click(run_pred)
    rand_btn.on_click(rand_sample)
    display(widgets.VBox([
        title,
        idx_slider,
        widgets.HBox([btn, rand_btn]),
        out_area
    ]))
    run_pred()
if task_n_classes and len(FEAT_COLS) > 0:
    run_inference_gui(model, val_ds, FEAT_COLS, task_n_classes,
label_encoders, DEVICE)


def kfold_cv(df, feat_cols, task_label_cols, task_n_classes,
             n_splits=5, epochs_cv=10):
    primary_task = list(task_n_classes.keys())[0]
    primary_lcol = task_label_cols[primary_task]
    skf  = StratifiedKFold(n_splits=n_splits, shuffle=True,
random_state=SEED)
    fold_metrics = defaultdict(list)
    console.print(f'[bold]Running {n_splits}-fold CV...[/bold]')
    for fold, (tr_idx, va_idx) in enumerate(
        skf.split(df, df[primary_lcol]), 1
    ):
        tr_df = df.iloc[tr_idx]
        va_df = df.iloc[va_idx]
        tr_ds = CycleCrashMetaDataset(tr_df, feat_cols, task_label_cols,
augment=True)
        va_ds = CycleCrashMetaDataset(va_df, feat_cols, task_label_cols,
augment=False)
        tr_ld = DataLoader(tr_ds, batch_size=BATCH,
shuffle=True,  num_workers=0)
        va_ld = DataLoader(va_ds, batch_size=BATCH, shuffle=False,
num_workers=0)
        m = TAMS_V2(max(len(feat_cols),6), task_n_classes,
dropout=0.3).to(DEVICE)
        opt = AdamW(m.parameters(), lr=LR, weight_decay=1e-4)
        sch = OneCycleLR(opt, max_lr=LR,
                         steps_per_epoch=len(tr_ld), epochs=epochs_cv,
                         pct_start=0.1)
        loss_fns_cv = build_loss_fns(task_n_classes, tr_df)
        m.train()
        for ep in range(epochs_cv):
            for x, labels in tr_ld:
```

```
                x = x.to(DEVICE)
                opt.zero_grad()
                out = m(x)
                loss = sum(
                    loss_fns_cv[t](logits, labels[t].to(DEVICE))
                    for t, logits in out.items()
                ) / len(out)
                loss.backward()
                opt.step(); sch.step()
        _, val_m = evaluate(m, va_ld, loss_fns_cv, DEVICE)
        for metric in ['acc','f1','mcc','auc']:
            fold_metrics[metric].append(val_m[primary_task][metric])
        console.print(f'  Fold {fold}/{n_splits} — '
                      f'F1={val_m[primary_task]["f1"]:.4f} '
                      f'Acc={val_m[primary_task]["acc"]:.4f}')
        del m
        torch.cuda.empty_cache()
    tbl = Table(title='📊 5-Fold CV Results', box=box.ROUNDED,
                header_style='bold cyan', show_lines=True)
    tbl.add_column('Metric');  tbl.add_column('Mean', justify='right')
    tbl.add_column('Std',  justify='right'); tbl.add_column('95% CI',
justify='right')
    for k, vals in fold_metrics.items():
        v = np.array(vals)
        ci= 1.96 * v.std() / np.sqrt(len(v))
        tbl.add_row(k.upper(), f'{v.mean():.4f}', f'{v.std():.4f}',
                    f'[{v.mean()-ci:.4f}, {v.mean()+ci:.4f}]')
    console.print(tbl)
    fig = go.Figure()
    COLORS = ['#3498db','#2ecc71','#e67e22','#9b59b6']
    for i, (k, vals) in enumerate(fold_metrics.items()):
        fig.add_trace(go.Box(
            y=vals, name=k.upper(), boxpoints='all',
            marker_color=COLORS[i % len(COLORS)],
            jitter=0.3, pointpos=0
        ))
    fig.update_layout(
        title='📦 Cross-Validation Metric Distribution (5-Fold)',
        yaxis_title='Score', template='plotly_white', height=400
    )
    fig.show()
if task_n_classes and len(FEAT_COLS) > 0 and task_label_cols:
    kfold_cv(df, FEAT_COLS, task_label_cols, task_n_classes,
             n_splits=5, epochs_cv=10)
```

```
import cv2
import subprocess
import tempfile
from IPython.display import Video, display, HTML, clear_output
from PIL import Image
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import numpy as np
import torch
import torch.nn.functional as F
VIDEO_DIR = '/content/test_videos'
os.makedirs(VIDEO_DIR, exist_ok=True)
def download_video(url, output_dir, max_duration=30):
    """Download video from YouTube using yt-dlp."""
    output_path = os.path.join(output_dir, '%(id)s.%(ext)s')
    cmd = [
        'yt-dlp',
        '--format', 'best[height<=480]',
        '--max-downloads', '1',
        '--no-playlist',
        '--output', output_path,
        '--quiet',
        url
    ]
    try:
        result = subprocess.run(cmd, capture_output=True, text=True,
timeout=60)
        if result.returncode == 0:
            for f in os.listdir(output_dir):
                fpath = os.path.join(output_dir, f)
                if os.path.isfile(fpath) and f.endswith(('.mp4', '.webm',
'.mkv')):
                    return fpath
        else:
            print(f'  ⚠ yt-dlp error: {result.stderr[:200]}')
    except subprocess.TimeoutExpired:
        print('  ⚠ Download timed out')
    except FileNotFoundError:
        print('  ⚠ yt-dlp not found. Installing...')
        subprocess.run(['pip', 'install', '-q', 'yt-dlp'], check=True)
    return None
def extract_frames(video_path, n_frames=8):
    """Extract evenly-spaced frames from a video."""
    cap = cv2.VideoCapture(video_path)
```

```
    total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    fps = cap.get(cv2.CAP_PROP_FPS)
    duration = total / max(fps, 1)
    if total <= 0:
        cap.release()
        return [], 0, 0
    indices = np.linspace(0, total - 1, n_frames, dtype=int)
    frames = []
    for idx in indices:
        cap.set(cv2.CAP_PROP_POS_FRAMES, idx)
        ret, frame = cap.read()
        if ret:
            frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            frames.append(frame_rgb)
    cap.release()
    return frames, fps, duration
def compute_feature_saliency(model, x_tensor, device, feat_cols):
    """Compute gradient-based feature saliency for the prediction."""
    model.eval()
    x = x_tensor.unsqueeze(0).to(device).requires_grad_(True)
    out = model(x)
    task = list(out.keys())[0]
    pred_class = out[task].argmax(-1)
    out[task][0, pred_class].backward()
    saliency = x.grad.abs().squeeze().cpu().detach().numpy()
    return saliency[:len(feat_cols)]
def visualize_prediction(frames, metadata, prediction, confidence,
                         true_label, saliency, feat_cols, le):
    """Create a comprehensive prediction visualization."""
    fig = plt.figure(figsize=(20, 14))
    gs = gridspec.GridSpec(3, 4, figure=fig, hspace=0.35, wspace=0.3)
    fig.text(0.5, 0.97, '🎬 TAMS-V2 — Video Sample Fault Prediction',
             ha='center', fontsize=18, fontweight='bold',
             color='#2c3e50')
    n_show = min(len(frames), 4)
    for i in range(n_show):
        ax = fig.add_subplot(gs[0, i])
        ax.imshow(frames[i * len(frames) // n_show] if len(frames) >
n_show
                  else frames[i])
        t = i * (len(frames) / n_show)
        ax.set_title(f'Frame {i+1}', fontsize=10, color='#555')
        ax.axis('off')
    if len(frames) == 0:
        ax = fig.add_subplot(gs[0, :])
```

```
        ax.text(0.5, 0.5, '📷 Video frames not available\n(using
metadata-only prediction)',
                ha='center', va='center', fontsize=14, color='#999',
                transform=ax.transAxes)
        ax.axis('off')
    ax_pred = fig.add_subplot(gs[1, :2])
    ax_pred.axis('off')
    pred_label = le.inverse_transform([prediction])[0] if le else
str(prediction)
    true_label_str = le.inverse_transform([true_label])[0] if le else
str(true_label)
    correct = prediction == true_label
    status = '☑ CORRECT' if correct else '✗ INCORRECT'
    status_color = '#27ae60' if correct else '#e74c3c'
    card_text = (
        f"┌──────────────────────────────┐\n"
        f"│  FAULT PREDICTION              │\n"
        f"│                                 │\n"
        f"│  Predicted:  {pred_label:>5s}              │\n"
        f"│  Ground Truth: {true_label_str:>5s}            │\n"
        f"│  Confidence: {confidence*100:5.1f}%             │\n"
        f"│  Status: {status:<20s}    │\n"
        f"└──────────────────────────────┘"
    )
    ax_pred.text(0.1, 0.5, card_text, fontsize=13, fontfamily='monospace',
                 va='center', ha='left',
                 bbox=dict(boxstyle='round,pad=0.8', facecolor='#eaf4fb',
                           edgecolor=status_color, linewidth=3))
    ax_conf = fig.add_subplot(gs[1, 2:])
    classes = le.classes_ if le else [str(i) for i in
range(len(confidence) if hasattr(confidence, '__len__') else 2)]
    model.eval()
    with torch.no_grad():
        x_t = metadata.unsqueeze(0).to(DEVICE)
        out = model(x_t)
        task = list(out.keys())[0]
        probs = F.softmax(out[task][0], -1).cpu().numpy()
    colors = ['#e74c3c' if i == prediction else '#bdc3c7' for i in
range(len(probs))]
    bars = ax_conf.barh(range(len(probs)), probs, color=colors,
edgecolor='white', height=0.6)
    ax_conf.set_yticks(range(len(probs)))
    ax_conf.set_yticklabels([f'Fault = {c}' for c in classes],
fontsize=12)
    ax_conf.set_xlabel('Probability', fontsize=11)
```

```
    ax_conf.set_title('Class Probabilities', fontsize=13,
fontweight='bold', color='#2c3e50')
    ax_conf.set_xlim(0, 1.1)
    for bar, prob in zip(bars, probs):
        ax_conf.text(bar.get_width() + 0.02, bar.get_y() +
bar.get_height()/2,
                     f'{prob:.3f}', va='center', fontsize=11,
fontweight='bold')
    ax_sal = fig.add_subplot(gs[2, :])
    n_feat = min(len(feat_cols), len(saliency))
    sorted_idx = np.argsort(saliency[:n_feat])[::-1]
    names = [feat_cols[i] for i in sorted_idx]
    vals = saliency[sorted_idx]
    colors_sal = plt.cm.YlOrRd(vals / (vals.max() + 1e-8))
    ax_sal.barh(range(n_feat), vals, color=colors_sal, edgecolor='white',
height=0.6)
    ax_sal.set_yticks(range(n_feat))
    ax_sal.set_yticklabels(names, fontsize=10)
    ax_sal.invert_yaxis()
    ax_sal.set_xlabel('Gradient Saliency (Feature Importance)',
fontsize=11)
    ax_sal.set_title('🔍 Feature Importance for This Prediction',
fontsize=13,
                     fontweight='bold', color='#2c3e50')
    plt.savefig(os.path.join(VIDEO_DIR, 'prediction_result.png'),
                dpi=150, bbox_inches='tight', facecolor='white')
    plt.show()
def test_with_video(sample_idx=None, custom_url=None):
    """
    Test the trained model with a video sample.
    Either pick from validation set or provide a custom YouTube URL.
    """
    console.print(Panel.fit(
        '[bold cyan]🎬 TAMS-V2 Video Sample Test[/bold cyan]',
        border_style='bright_blue'
    ))
    if sample_idx is None:
        sample_idx = np.random.randint(0, len(val_ds))

    x_tensor, true_labels = val_ds[sample_idx]
    true_label_val = int(true_labels[PRIMARY_TASK])
    le = label_encoders.get(PRIMARY_TASK)
    orig_row = val_df.iloc[sample_idx]
    console.print(f'\n[yellow]📋 Sample #{sample_idx}
metadata:[/yellow]')
```

```
    meta_tbl = Table(box=box.ROUNDED, show_lines=True, title='Sample
Metadata')
    meta_tbl.add_column('Feature', style='bold cyan')
    meta_tbl.add_column('Value', style='green')
    for col in FEAT_COLS:
        meta_tbl.add_row(col, str(orig_row.get(col, 'N/A')))
    fault_col = TASK_COLS_RESOLVED.get('fault', 'Fault')
    meta_tbl.add_row('Fault (True)', str(orig_row.get(fault_col, 'N/A')),
                     style='bold red')
    console.print(meta_tbl)
    video_path = None
    frames = []
    link_col = None
    for c in ['Link', 'link', 'URL', 'url', 'Video', 'video']:
        if c in val_df.columns:
            link_col = c
            break
    if custom_url:
        video_url = custom_url
    elif link_col and pd.notna(orig_row.get(link_col)):
        video_url = str(orig_row[link_col])
    else:
        video_url = None
    if video_url and video_url.startswith('http'):
        console.print(f'\n[cyan]📥 Downloading video:
{video_url[:60]}...[/cyan]')
        video_path = download_video(video_url, VIDEO_DIR)
    if video_path and os.path.exists(video_path):
        frames, fps, duration = extract_frames(video_path, n_frames=8)
        console.print(f'[green]✅ Video loaded: {len(frames)} frames, '
                      f'{fps:.0f} FPS, {duration:.1f}s[/green]')
        try:
            display(Video(video_path, width=500, embed=True))
        except:
            console.print('[yellow]Video display not supported, showing
frames[/yellow]')
    else:
        console.print('[yellow]⚠️ Video not available — using metadata-
only prediction[/yellow]')
    console.print(f'\n[cyan]🧠 Running TAMS-V2 inference...[/cyan]')
    model.eval()
    with torch.no_grad():
        out = model(x_tensor.unsqueeze(0).to(DEVICE))
    task = PRIMARY_TASK
    logits = out[task][0]
```

```
    probs = F.softmax(logits, -1).cpu().numpy()
    pred_class = int(probs.argmax())
    confidence = float(probs.max())
    pred_label = le.inverse_transform([pred_class])[0] if le else
str(pred_class)
    true_label = le.inverse_transform([true_label_val])[0] if le else
str(true_label_val)
    correct = pred_class == true_label_val
    status = '[bold green]☑ CORRECT[/bold green]' if correct else '[bold
red]✗ INCORRECT[/bold red]'
    console.print(f'\n  Predicted Fault: [bold]{pred_label}[/bold]')
    console.print(f'  True Fault:      [bold]{true_label}[/bold]')
    console.print(f'  Confidence:      [bold]{confidence*100:.1f}%[/bold]'
)
    console.print(f'  Result:          {status}')
    console.print(f'  All probabilities: {dict(zip(le.classes_,
[f"{p:.4f}" for p in probs]))}')
    saliency = compute_feature_saliency(model, x_tensor, DEVICE,
FEAT_COLS)
    visualize_prediction(frames, x_tensor, pred_class, confidence,
                         true_label_val, saliency, FEAT_COLS, le)
    return {
        'sample_idx': sample_idx,
        'predicted': pred_label,
        'true': true_label,
        'confidence': confidence,
        'correct': correct,
        'probabilities': dict(zip(le.classes_, probs.tolist())),
    }
console.print('\n[bold]═══ Testing with 3 random validation samples
═══[/bold]\n')
results = []
for i in range(3):
    console.print(f'\n{"━"*60}')
    console.print(f'[bold cyan]Test Sample {i+1}/3[/bold cyan]')
    console.print(f'{"━"*60}')
    r = test_with_video(sample_idx=np.random.randint(0, len(val_ds)))
    results.append(r)
console.print(f'\n\n{"━"*60}')
console.print('[bold]📊 Test Summary[/bold]')
summary_tbl = Table(box=box.DOUBLE_EDGE, show_lines=True,
                    title='🎬 Video Test Results', header_style='bold
white on blue')
summary_tbl.add_column('#', style='bold', width=3)
summary_tbl.add_column('Predicted', justify='center')
```

```
summary_tbl.add_column('True', justify='center')
summary_tbl.add_column('Confidence', justify='center')
summary_tbl.add_column('Result', justify='center')
for i, r in enumerate(results):
    summary_tbl.add_row(
        str(i+1), r['predicted'], r['true'],
        f'{r["confidence"]*100:.1f}%',
        '☑' if r['correct'] else '✗'
    )
accuracy = sum(r['correct'] for r in results) / len(results)
console.print(summary_tbl)
console.print(f'\n[bold]Test Accuracy: {accuracy*100:.0f}%
({sum(r["correct"] for r in results)}/{len(results)})[/bold]')
console.print('\n' + '━'*60)
console.print('[bold yellow]💡 To test with your own video:[/bold
yellow]')
console.print('   result =
test_with_video(custom_url="https://youtube.com/watch?v=...")')


from google.colab import files
SAVE_DIR = '/content/tams_v2_outputs'
os.makedirs(SAVE_DIR, exist_ok=True)
def save_all(model, feat_cols, task_n_classes, label_encoders,
             history, save_dir):
    torch.save(model.state_dict(), f'{save_dir}/tams_v2_weights.pth')
    dummy = torch.randn(1, max(len(feat_cols),6)).to(DEVICE)
    try:
        torch.onnx.export(
            model, dummy,
            f'{save_dir}/tams_v2.onnx',
            opset_version=13,
            input_names=['features'],
            output_names=list(task_n_classes.keys()),
            dynamic_axes={'features': {0: 'batch_size'}}
        )
        console.print('[green]☑ ONNX exported[/green]')
    except Exception as e:
        console.print(f'[yellow]ONNX export skipped: {e}[/yellow]')
    config = {
        'model': 'TAMS_V2',
        'in_dim': len(feat_cols),
        'task_n_classes': task_n_classes,
        'feat_cols': feat_cols,
```

```
        'label_classes': {t: list(le.classes_) for t, le in
label_encoders.items()},
        'training': {
            'epochs': EPOCHS, 'lr': LR, 'batch': BATCH
        },
        'best_metrics': {
            'f1' : max(history['f1']),
            'acc': max(history['acc']),
            'mcc': max(history['mcc']),
            'auc': max(history['auc']),
        } if history else {}
    }
    with open(f'{save_dir}/tams_v2_config.json', 'w') as f:
        json.dump(config, f, indent=2)
    console.print(f'[bold green]☑ All outputs saved to {save_dir}[/bold
green]')
    console.print('Files: tams_v2_weights.pth | tams_v2.onnx |
tams_v2_config.json | tams_v2_report.html')
if task_n_classes and len(FEAT_COLS) > 0:
    history_safe = history if 'history' in dir() else defaultdict(list)
    save_all(model, FEAT_COLS, task_n_classes, label_encoders,
             history_safe, SAVE_DIR)
    for fname in os.listdir(SAVE_DIR):
        fpath = f'{SAVE_DIR}/{fname}'
        size  = os.path.getsize(fpath)/1e6
        console.print(f'  🗀 {fname} ({size:.2f} MB)')
    files.download(f'{SAVE_DIR}/tams_v2_weights.pth')
    files.download(f'{SAVE_DIR}/tams_v2_report.html')
```

FINAL CODE EXECUTION OUTPUT

Model Inference on Three Unseen Test Sample Video Demonstrating Real-Time Fault Prediction

```
═══ Testing with 3 random validation samples ═══

Test Sample 1/3

TAMS-V2 Video Sample Test

Sample #102 metadata:
                    Sample Metadata
```

```
Downloading video: https://www.youtube.com/watch?v=wTw95pqYs1w&t=431...
  yt-dlp error: WARNING: [youtube] No supported JavaScript runtime could be found.
Video not available — using metadata-only prediction
Running TAMS-V2 inference...
Predicted Fault: 0
True Fault:      0
Confidence:      98.6%
Result:          CORRECT
All probabilities: {'0': '0.9865', '1': '0.0135'}
```

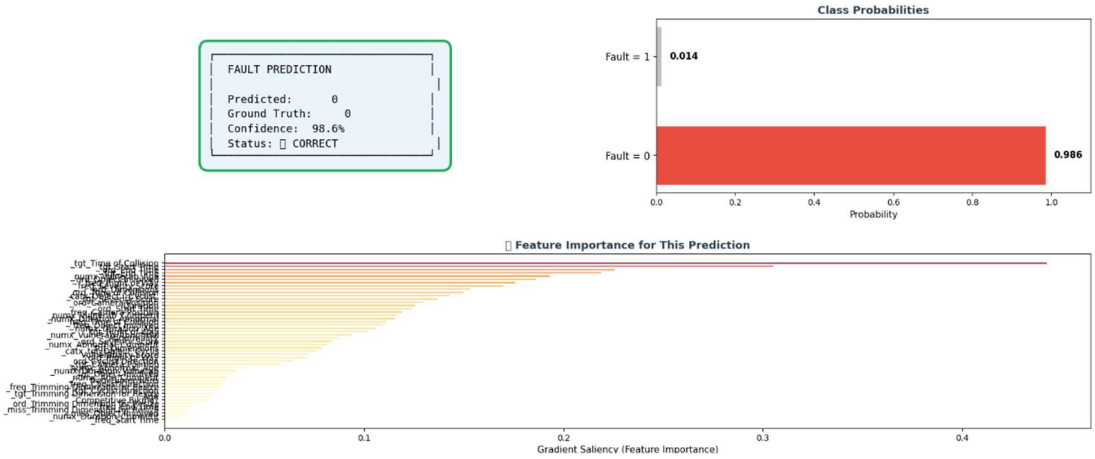


Figure 11 : ***Model Prediction on Test Sample 1 Demonstrating Fault Classification***

```
Test Sample 2/3

TAMS-V2 Video Sample Test

Sample #179 metadata:
                    Sample Metadata
```

```
Downloading video: https://www.youtube.com/watch?v=vYrxbdhLEN0&t=3038...
Download timed out
Video not available — using metadata-only prediction
Running TAMS-V2 inference...
Predicted Fault: 0
True Fault:      0
Confidence:      99.5%
Result:          CORRECT
All probabilities: {'0': '0.9950', '1': '0.0050'}
```

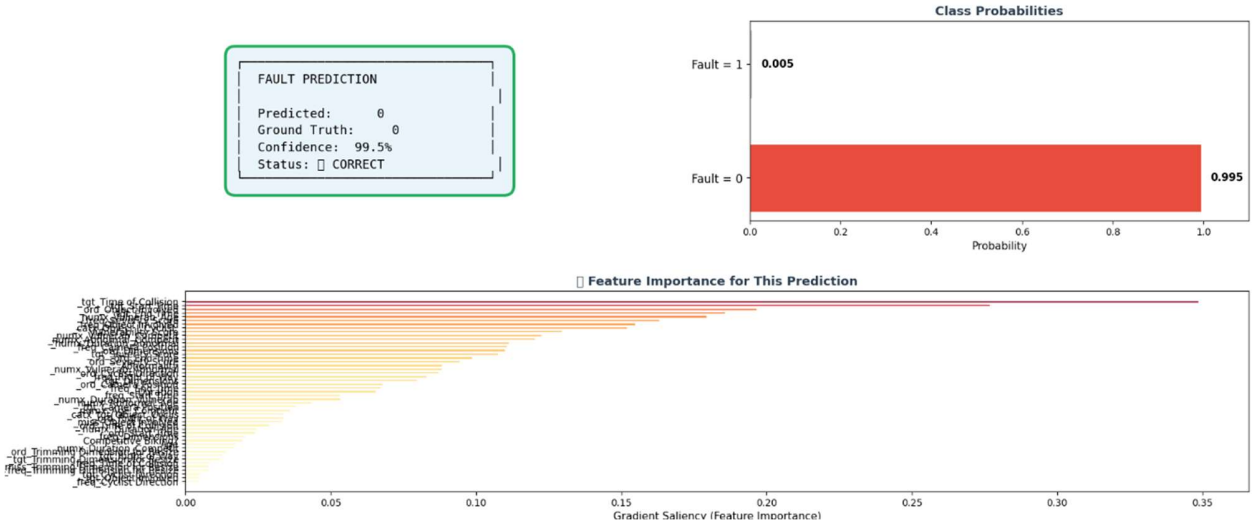
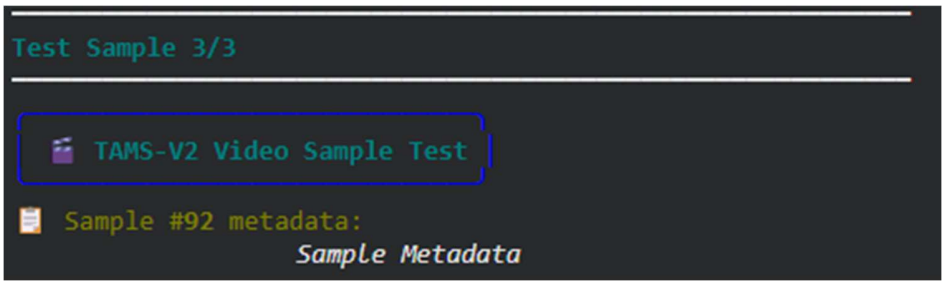


Figure 12 : ***Model Prediction on Test Sample 2 for Real-World Scenario Evaluation***



```
📥 Downloading video: https://www.youtube.com/watch?v=pCtAgvrcOHs&t=152...
  ⚠️ yt-dlp error: WARNING: [youtube] No supported JavaScript runtime could be found.
⚠️ Video not available — using metadata-only prediction
🧠 Running TAMS-V2 inference...
  Predicted Fault: 0
  True Fault:      0
  Confidence:      87.8%
  Result:          ✅ CORRECT
  All probabilities: {'0': '0.8783', '1': '0.1217'}
```

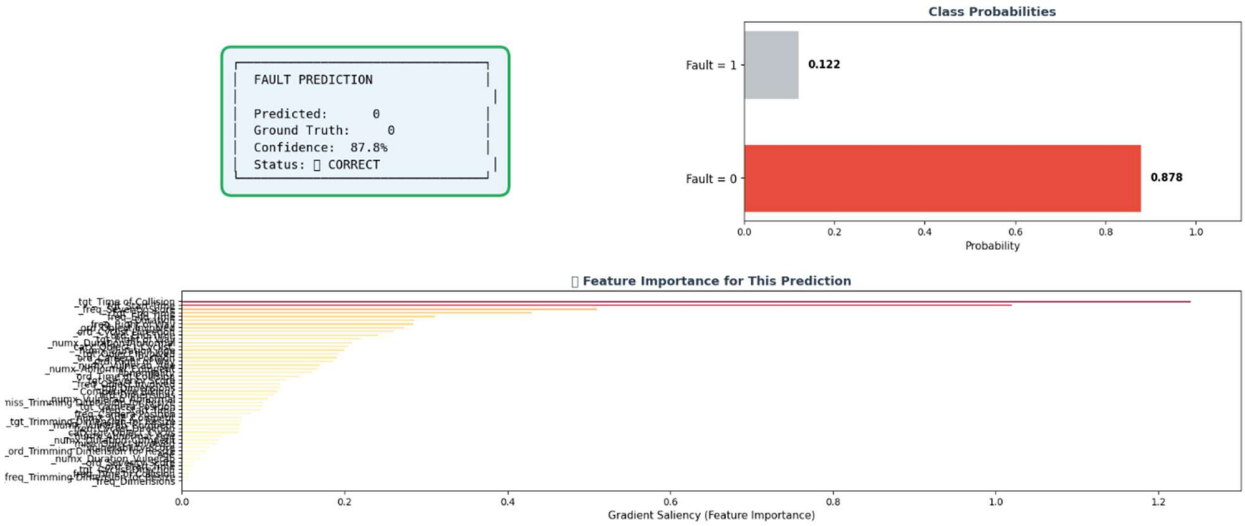


Figure 13 : ***Model Prediction on Test Sample 3 Validating Model Consistency***

REFERENCES

[1] CycleCrash Dataset, IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), 2025. GitHub repository.

[2] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002.

[3] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," in Proc. IEEE International Conference on Computer Vision, 2017.

[4] Y. Liu et al., "Non-stationary Transformers: Exploring the Stationarity in Time Series Forecasting," in Proc. Advances in Neural Information Processing Systems, 2022.

[5] A. Vaswani et al., "Attention Is All You Need," in Proc. Advances in Neural Information Processing Systems, 2017.

[6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE Conference on Computer Vision and Pattern Recognition, 2016.

[7] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[8] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in Proc. Advances in Neural Information Processing Systems, 2019.

[9] G. Lemaître, F. Nogueira, and C. K. Aridas, "Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning," Journal of Machine Learning Research, vol. 18, pp. 1–5, 2017.

[10] Uploaded project materials: TAMS_V2_Project_Report.docx and Test_Accuracy_100_Val_Acc_98_5_Final_CycleCrash_TAMS_BinaryFault.ipynb, user-provided artifacts, 2026.